МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационных систем и технологий
Специальность 6-05-0611-01 «Информационные системы и технологии»

Дисциплина Операционные системы

Лабораторная работа № 3
УПРАВЛЕНИЕ ПРОЦЕССАМИ

Студент 3 курса 2 группы
Марушко Т. Ф.
Проверил: Ассистент Савельева М. Г.

Минск, 2026

Содержание

Введение....3
1 Выполнение заданий на ОС *Linux*....4
1.1 Подготовка среды....4
1.2 Уровень 4–5: базовые операции с процессами....4
1.3 Уровень 7–8: *IPC* через анонимный канал....8
1.4 Уровень 9–10: обработка сигналов *SIGINT*....10
1.5 Вывод по разделу....11
2 Выполнение заданий на ОС *Windows*....12
2.1 Подготовка среды и сборка....12
2.2 Уровень 4–5: создание процессов (*CreateProcess*)....12
2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*....18
2.4 Уровень 9–10: передача через *STARTUPINFO*....20
2.5 Вывод по разделу....22
Заключение....23
ПРИЛОЖЕНИЕ А. Исходные тексты программ....24

Введение

Цель работы – получить навыки управления процессами в *Windows* и *Linux*: создание, контроль, обмен данными между процессами, передача через *STARTUPINFO*.

Задачи:

– написать *Lab*-03*x*: цикл 500 мс, вывод *PID*, аргумент или *ITER_NUM*;

– реализовать *Lab*-03*a*: три дочерних процесса *CreateProcess* разными способами;

– реализовать *Lab*-03*b*: глобальная и локальная *ITER_NUM*;

– написать *Lab*-03*c*: список процессов через *Tool Help Library* (*PID*, *PPID*, имя);

– реализовать *Lab*-03*d-client* / *Lab*-03*d-server*: поиск чисел, взаимно простых с *M*, через анонимный канал и *STARTUPINFO*;

– реализовать *Lab*-03*hello-parent* / *Lab*-03*hello-child*: передача строки через *STARTUPINFO* (*lpTitle* + *CREATE_NEW_CONSOLE*);

– написать аналоги для *Linux*: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig* (устойчивость к *SIGINT*);

– сопоставить результаты запусков и способы взаимодействия процессов в обеих операционных системах.

Работа выполнена в виртуальных машинах *Arch Linux ARM* и *Windows* 11 *ARM*. Для *Linux* использован компилятор *GCC*, для *Windows* – *MSVC*. В работе сопоставлены способы запуска дочерних процессов, передачи параметров и обмена результатами через анонимный канал.

1 Выполнение заданий на ОС *Linux*

1.1 Подготовка среды

Linux запущен в *VMware Fusion* (*Arch ARM*, *KDE Plasma*). Пользователь *User*-6*d*4*ec*851. Исходники в ~/*OS*3. Сборка показана на рисунке 1.1.

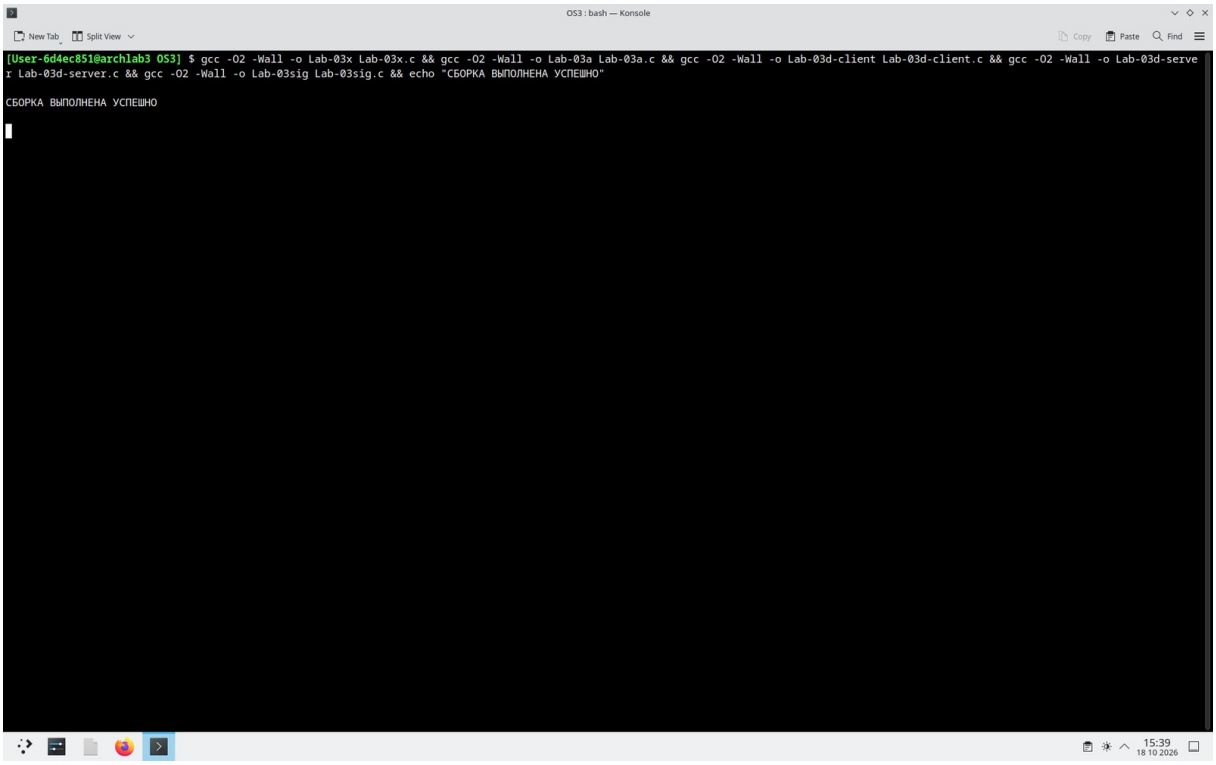


Рисунок 1.1 – Сборка *Linux*-приложений: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*sig*

Для каждой программы выполнена компиляция командой *gcc* с параметрами *-O*2 *-Wall*. После последовательной сборки пяти программ получено сообщение «СБОРКА ВЫПОЛНЕНА УСПЕШНО». Исходный текст *Lab*-03*x* приведён в листинге А.1.

1.2 Уровень 4–5: базовые операции с процессами

Для проверки передачи параметров выполнены три запуска *Lab*-03*x*: с аргументом командной строки, с переменной окружения *ITER_NUM* и без обоих источников. На каждой итерации программа выводит *PID* и номер итерации, затем выполняет задержку 500 мс.

Lab-03*x* запущен с аргументом (5 итераций), через *ITER_NUM*=7, без параметров (ошибка). Результаты на рисунках 1.2, 1.3, 1.4.

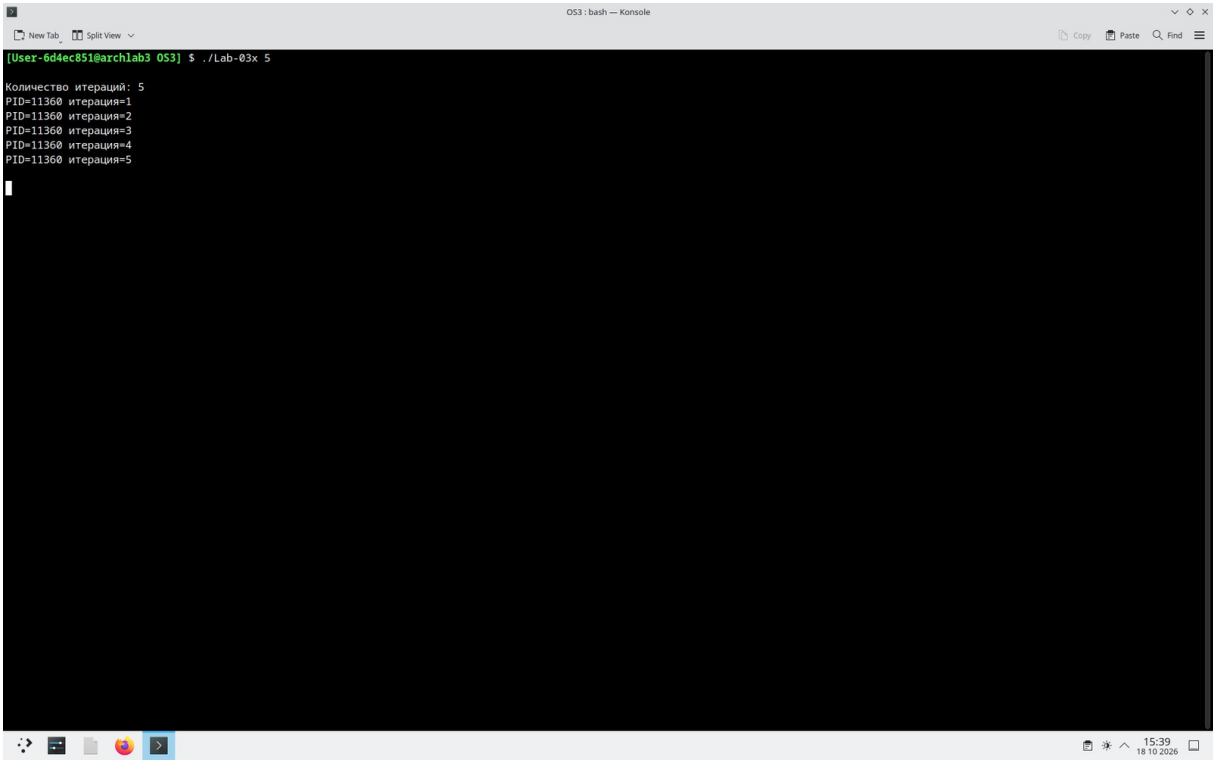


Рисунок 1.2 – *Lab*-03*x* с аргументом: 5 итераций, вывод *PID*

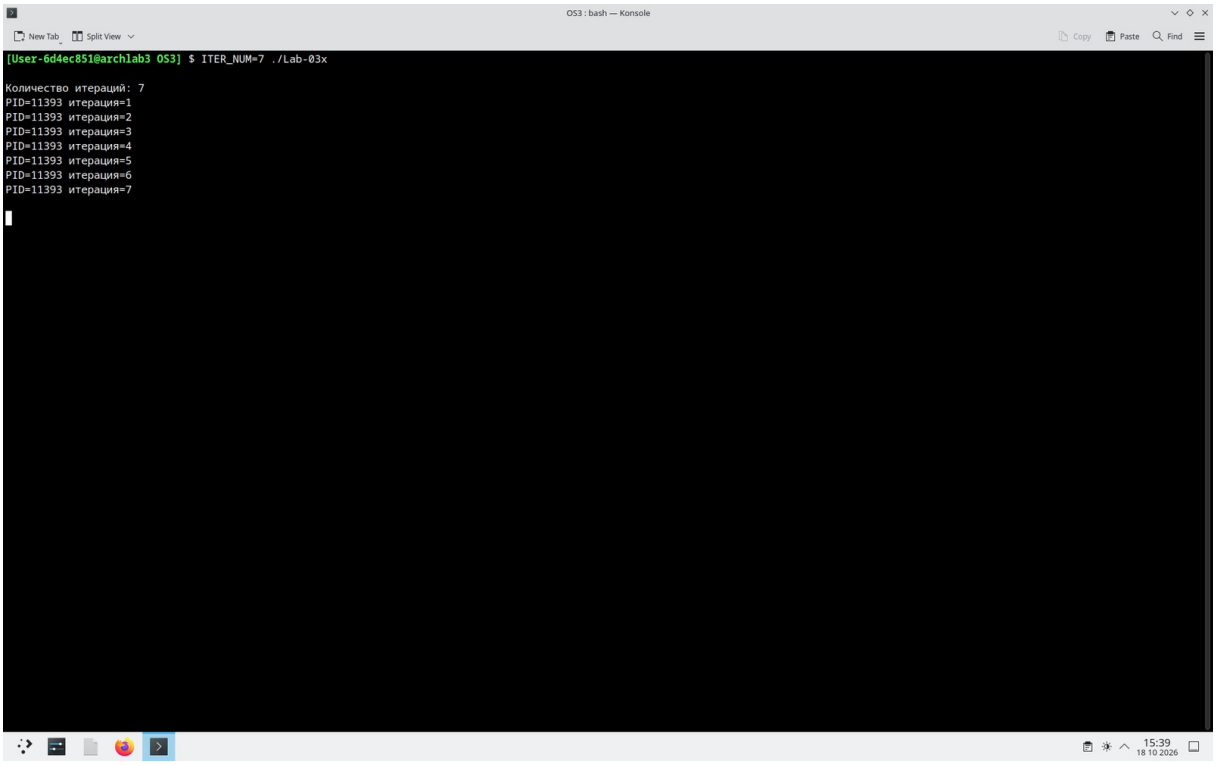


Рисунок 1.3 – *Lab*-03*x* через *ITER_NUM*=7: 7 итераций

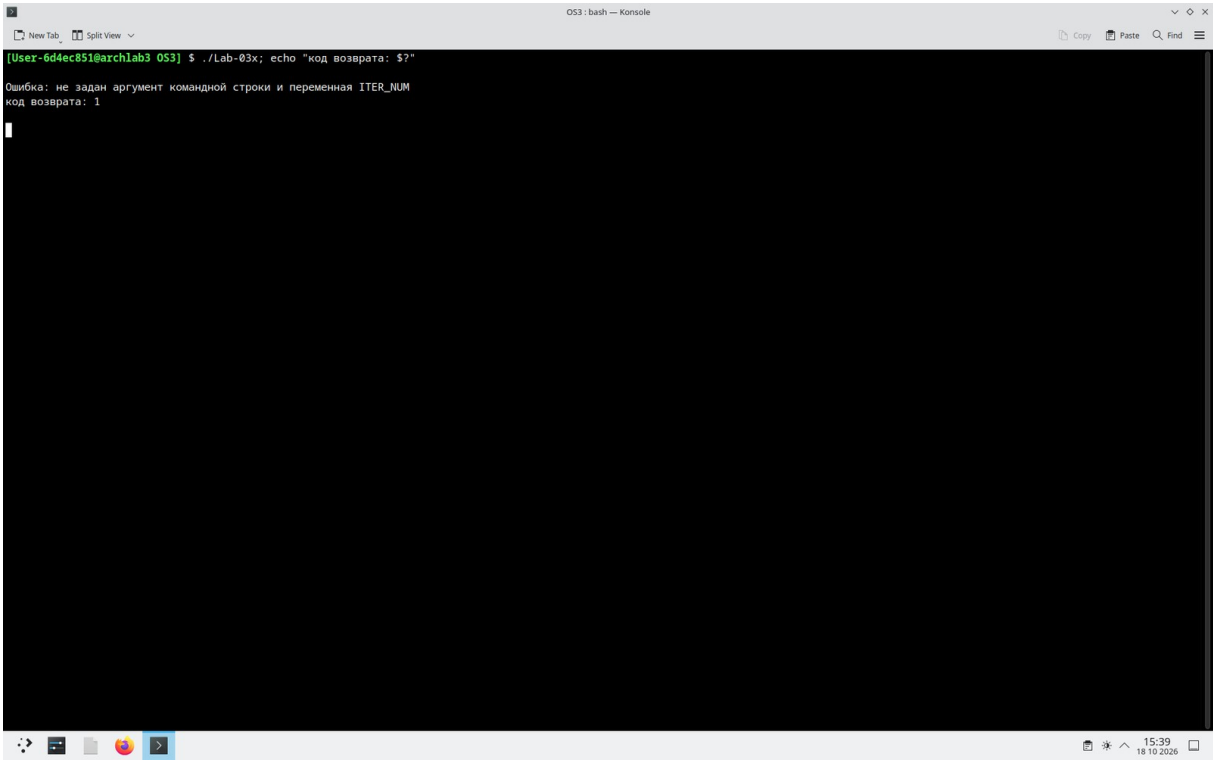


Рисунок 1.4 – *Lab*-03*x* без параметров: ошибка, код возврата 1

В *Lab*-03*a* два дочерних процесса создаются через *fork* и загружают *Lab*-03*x* вызовом *execl*. Первому передаётся аргумент 5, для второго родитель устанавливает *ITER_NUM*=8. После запуска обоих процессов родитель ожидает их через *waitpid*. Исходный текст приведён в листинге А.2. На рисунке 1.5 показаны чередующийся вывод дочерних процессов и их завершение с кодом 0.

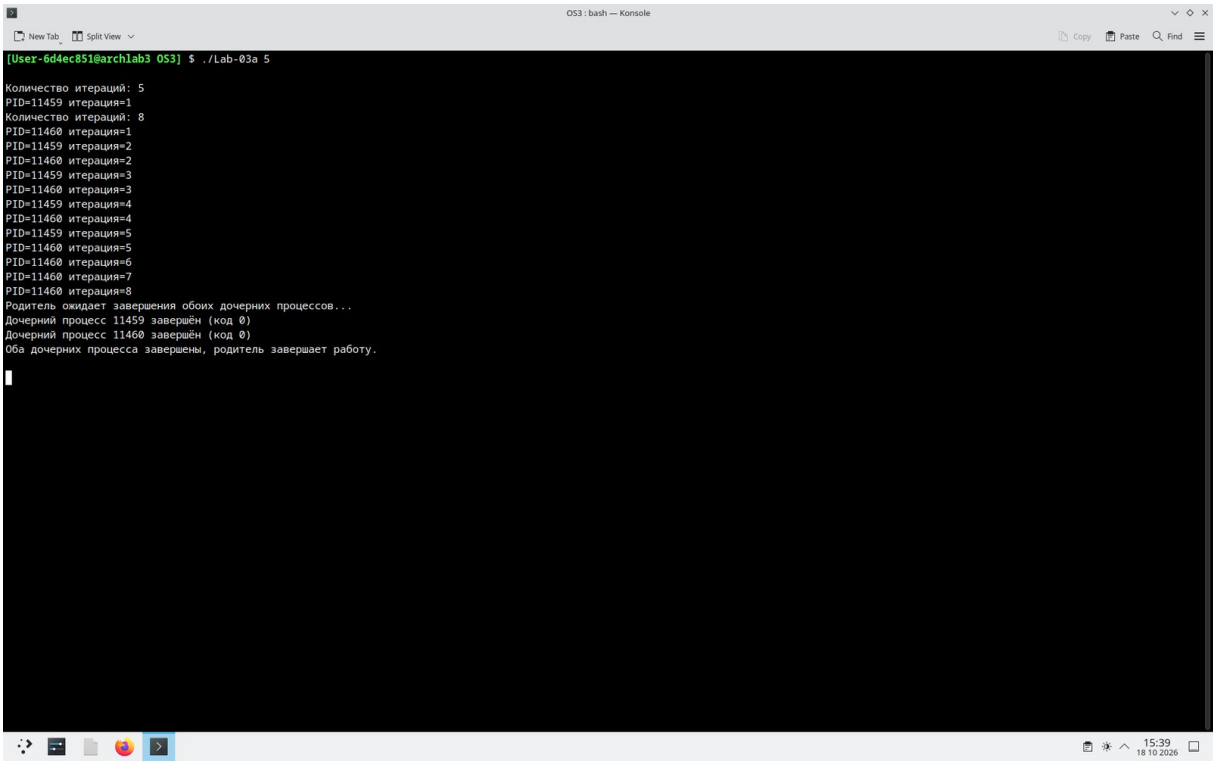


Рисунок 1.5 – *Lab*-03*a*: параллельное выполнение двух *Lab*-03*x* и ожидание их завершения

Для изучения родительских связей использована команда *ps -ef*. Дерево процессов показано на рисунке 1.6. Файл */proc/<PID>/status* содержит состояние процесса, *PID*, *PPID*, число потоков и объём резидентной памяти; соответствующий вывод и содержимое каталога */proc/<PID>* приведены на рисунке 1.7. Исходные и исполняемые файлы в рабочем каталоге показаны на рисунке 1.8.

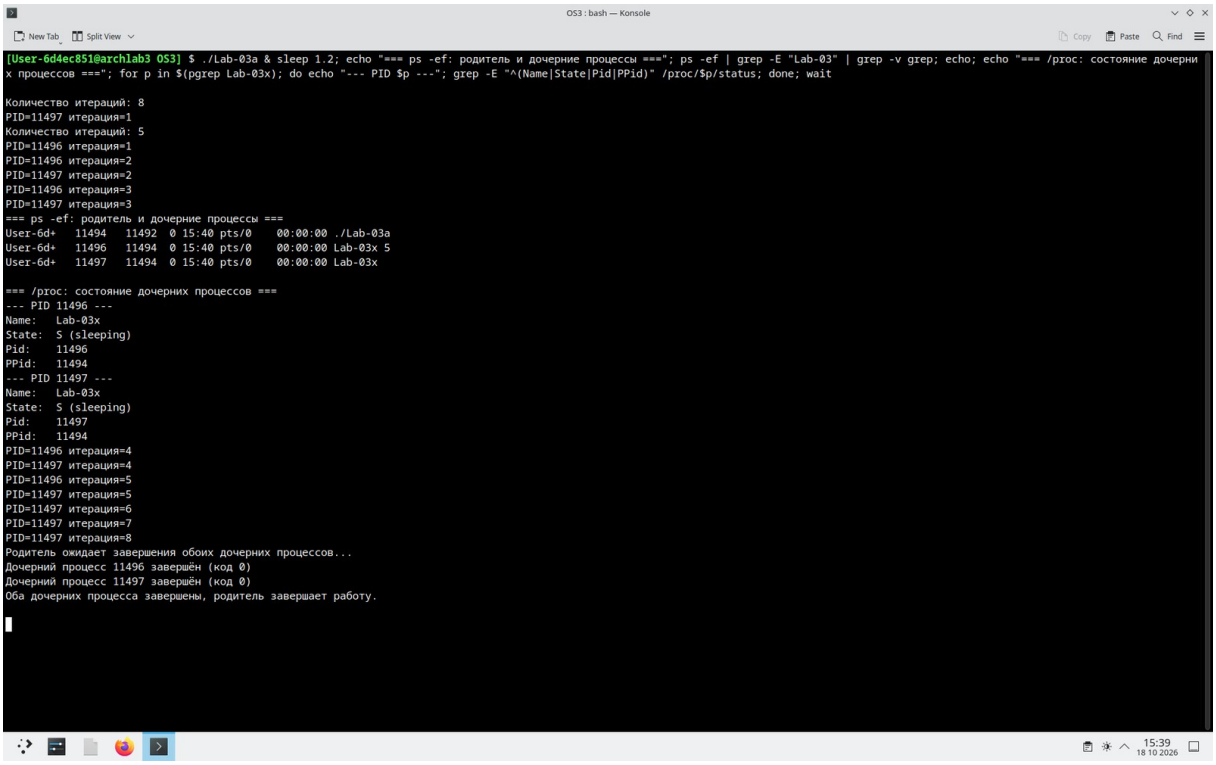


Рисунок 1.6 – Дерево процессов: *ps -ef* показывает родительские связи (*PPID*)

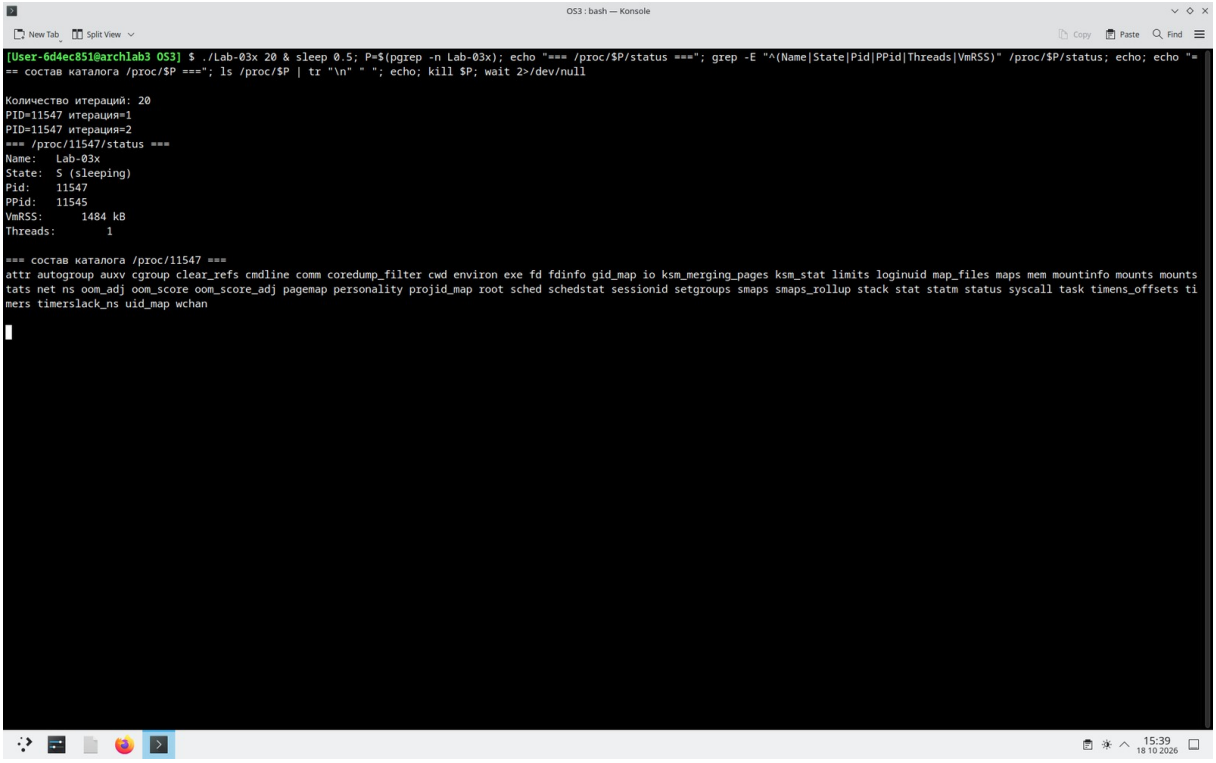


Рисунок 1.7 – Состояние процесса и содержимое каталога */proc/<PID>*

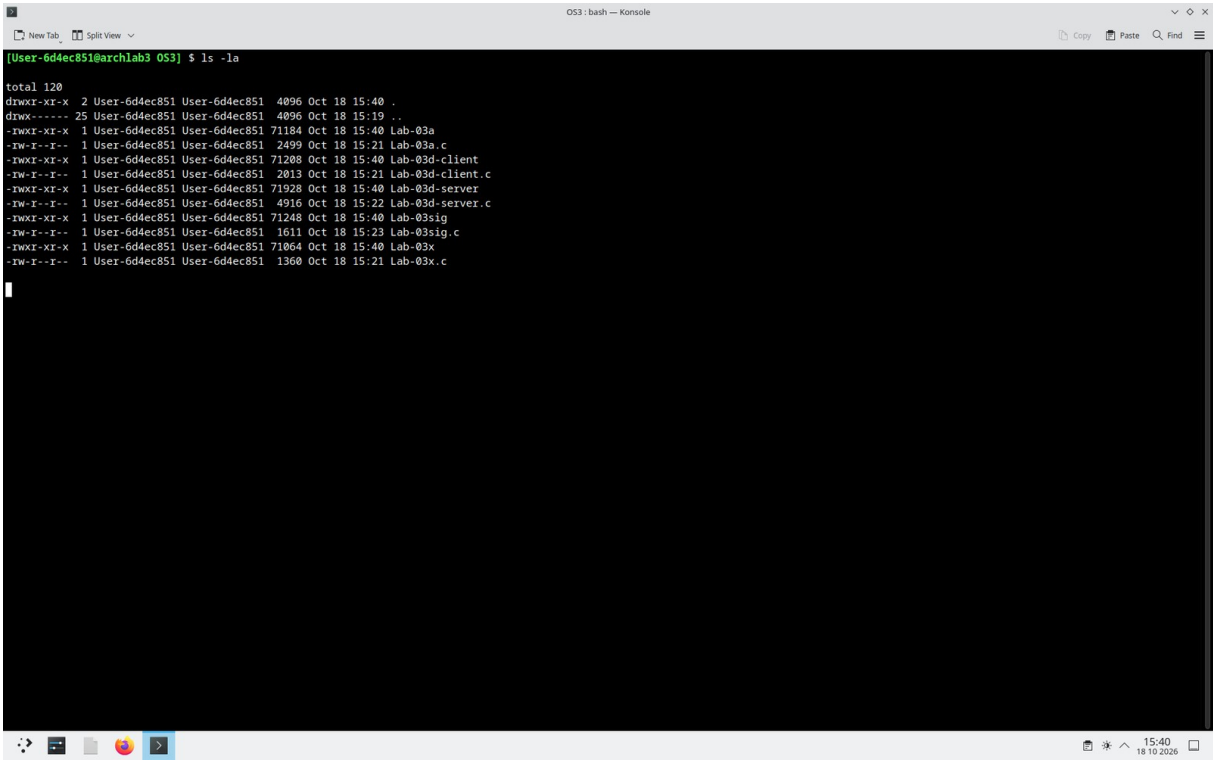


Рисунок 1.8 – Исходные и исполняемые файлы в рабочем каталоге ~/*OS*3

Запуски показали приоритет аргумента командной строки и возможность получения числа итераций из *ITER_NUM*. При отсутствии обоих источников *Lab*-03*x* завершился с кодом 1. Ожидание дочерних процессов в *Lab*-03*a* организовано с помощью *waitpid*.

1.3 Уровень 7–8: *IPC* через анонимный канал

Для поиска чисел, взаимно простых с *M*, разработаны *Lab*-03*d-client* и *Lab*-03*d-server*. Клиент проверяет условие НОД(*n*, *M*) = 1 для каждого *n* из [*L*, *R*] и передаёт найденные числа через стандартный вывод. Сервер создаёт *pipe*, делит диапазон на *K* поддиапазонов и запускает клиентов через *fork* и *execl*; перед запуском стандартный вывод каждого клиента перенаправляется в канал вызовом *dup*2. Исходные тексты приведены в листингах А.3 и А.4. Отдельный запуск клиента показан на рисунке 1.9, запуски сервера с тремя и четырьмя процессами – на рисунках 1.10 и 1.11.

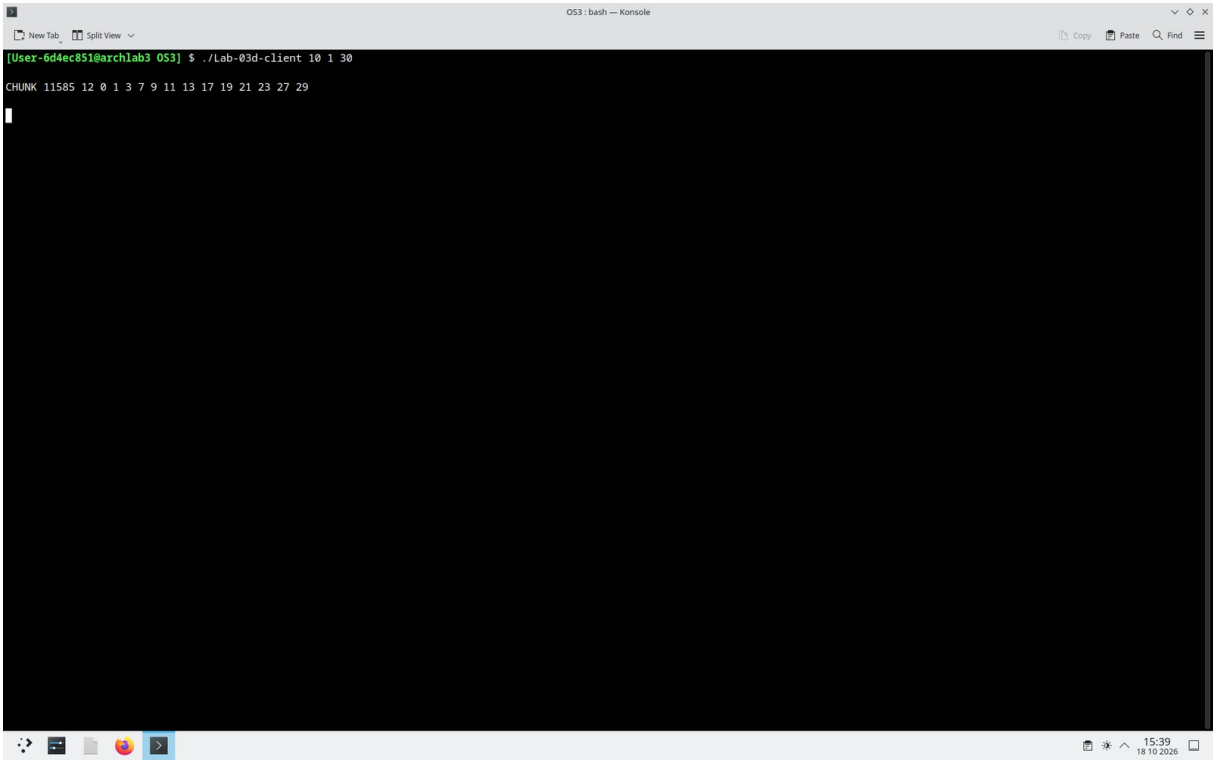


Рисунок 1.9 – *Lab*-03*d-client*: *M*=10, диапазон 1–30, найдены числа, вывод в канал

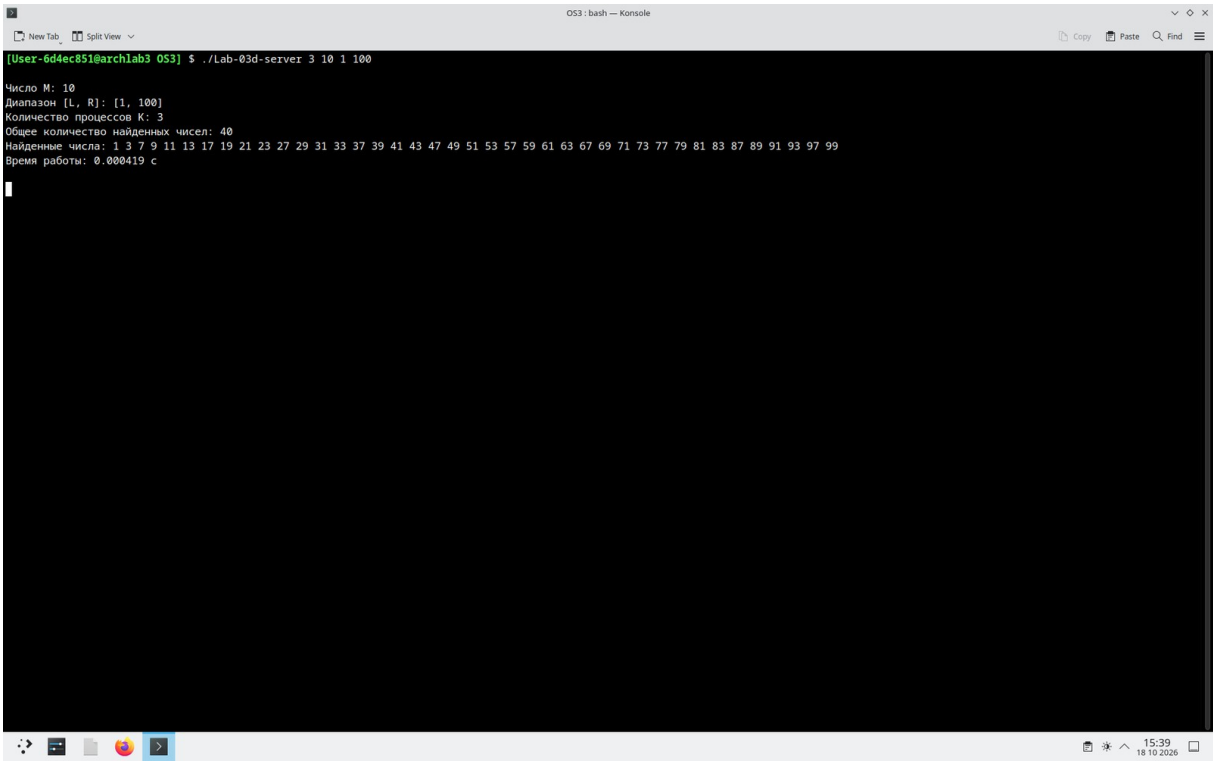


Рисунок 1.10 – *Lab*-03*d-server*: *K*=3, *M*=10, диапазон [1, 100], найдено 40 чисел

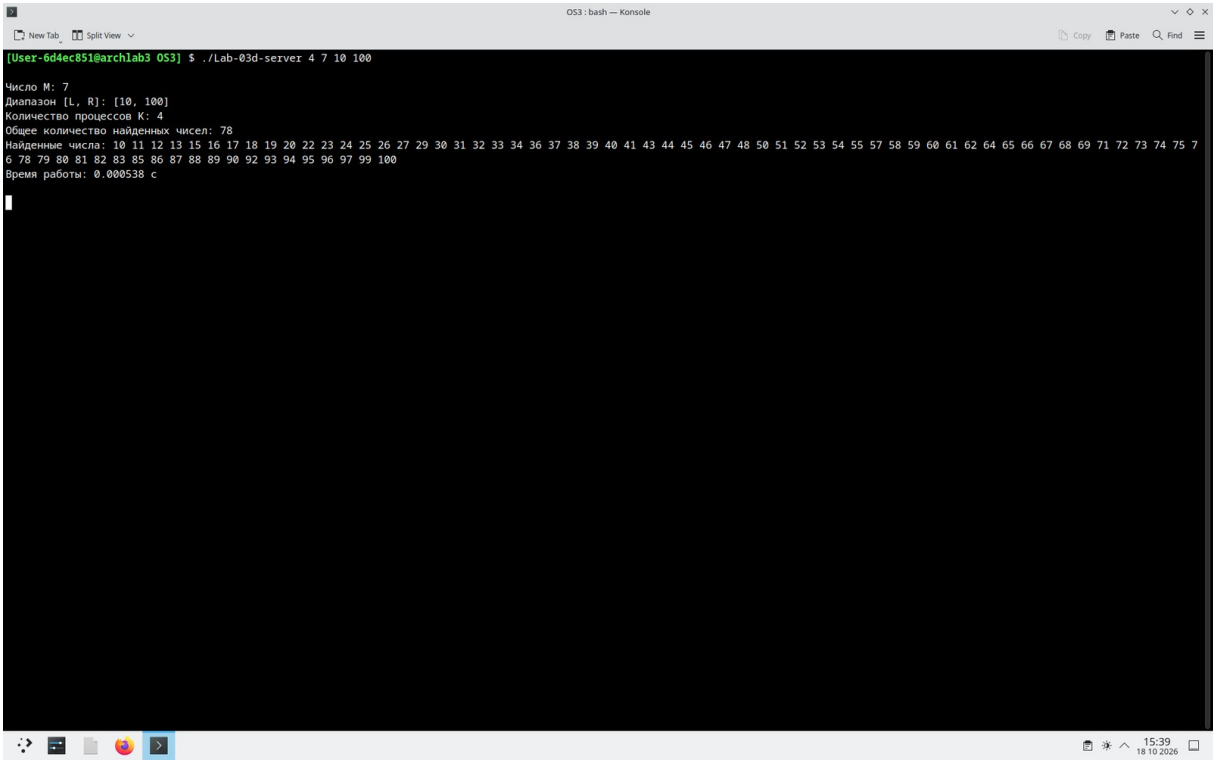


Рисунок 1.11 – *Lab*-03*d-server K*=4, *M*=7, 10–100: результат с 4 поддиапазонами

При *M* = 10 в диапазоне [1, 100] сервер получил 40 чисел, при *M* = 7 в диапазоне [10, 100] – 78. Сервер читает сообщения из канала и ожидает завершения клиентов через *waitpid*. Числа выводятся в порядке получения сообщений; отдельная сортировка в программе не выполняется. Продолжительность измеряется с помощью *clock_gettime* с *CLOCK_MONOTONIC*.

1.4 Уровень 9–10: обработка сигналов *SIGINT*

Для сохранения работы после *SIGINT* в *Lab*-03*sig* установлен обработчик сигнала. Он увеличивает счётчик, повторно устанавливает обработчик и выводит уведомление через *write*. Основной цикл продолжает выполнение после обработки сигнала. Исходный текст приведён в листинге А.5. На рисунке 1.12 показана отправка двух *SIGINT* командой *kill -INT* и последующее завершение через *kill* -9.

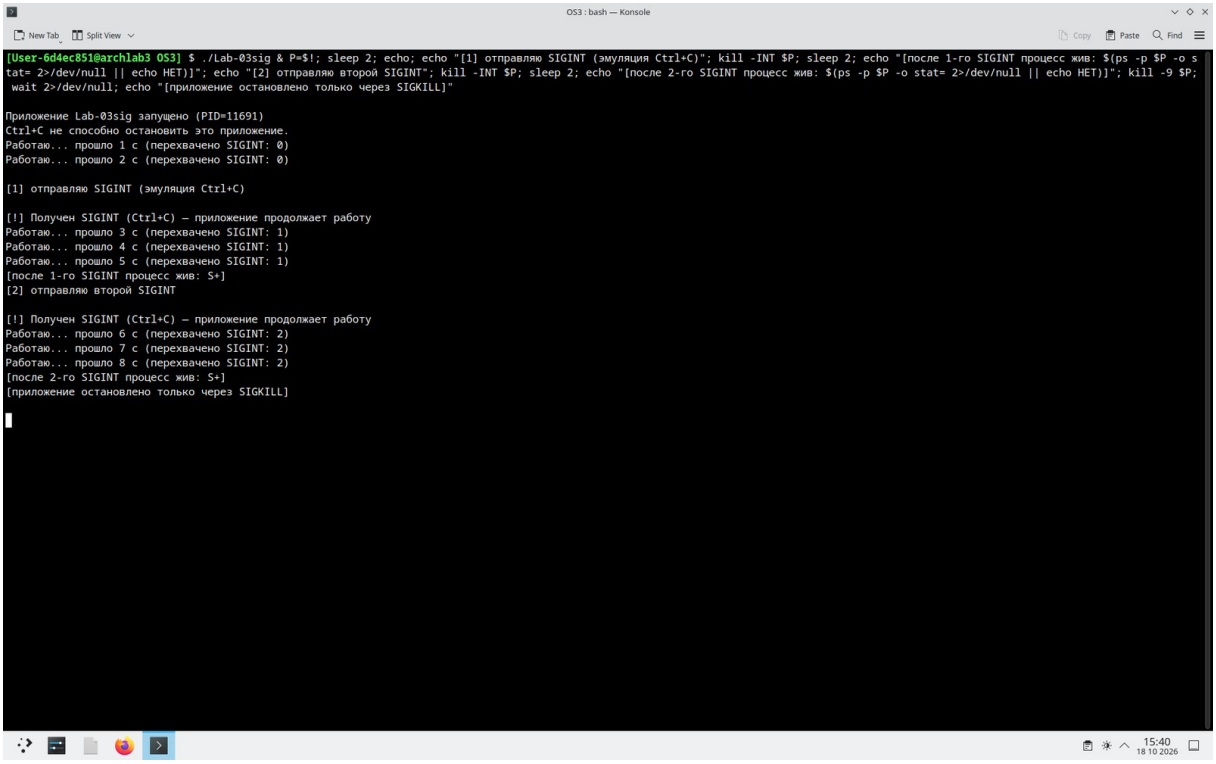


Рисунок 1.12 – *Lab*-03*sig*: продолжение работы после двух *SIGINT* и завершение через *SIGKILL*

После каждого из двух *SIGINT* процесс продолжил вывод состояния, а счётчик обработанных сигналов увеличился. В данном запуске процесс завершён сигналом *SIGKILL*. Обработчик *SIGTERM* в программе не установлен, поэтому этот сигнал также сохраняет стандартное действие завершения.

1.5 Вывод по разделу

В *Linux* создание процесса отделено от загрузки программы: *fork* создаёт дочерний процесс, а *execl* заменяет его исполняемый образ. Параметры передаются через аргументы и окружение, результаты вычислений – через *pipe* с перенаправлением *stdout*. Ожидание через *waitpid* позволяет родителю завершиться после дочерних процессов. Обработка *SIGINT* продемонстрировала продолжение основного цикла после поступления сигнала.

2 Выполнение заданий на ОС *Windows*

2.1 Подготовка среды и сборка

Программы для *Windows* собраны в виртуальной машине *Windows* 11 *ARM* компилятором *MSVC* для *ARM*64. Заголовочный файл *console_ru.h* выполняет перекодирование консольного вывода из *UTF*-8 в *OEM* 866. Его текст приведён в листинге А.14. Запуск сборки показан на рисунке 2.1.

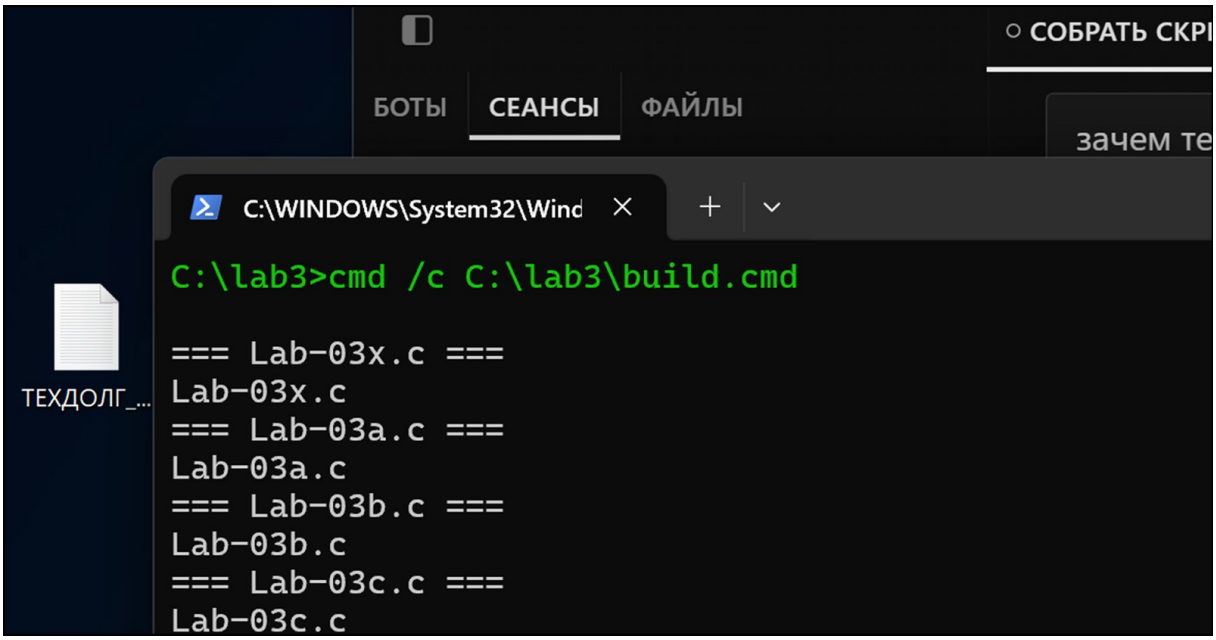


Рисунок 2.1 – Сборка 8 *Windows*-приложений: *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*b*, *Lab*-03*c*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*hello-parent*, *Lab*-03*hello-child*

Пакетный файл *build.cmd* настраивает окружение вызовом *vcvarsall.bat arm*64 и компилирует программы с ключами /*nologo* /*W*3 /*O*2 /*utf*-8. Для проверки работы используются полученные исполняемые файлы *Lab*-03*x*, *Lab*-03*a*, *Lab*-03*b*, *Lab*-03*c*, *Lab*-03*d-client*, *Lab*-03*d-server*, *Lab*-03*hello-parent* и *Lab*-03*hello-child*.

2.2 Уровень 4–5: создание процессов (*CreateProcess*)

В *Lab*-03*x* реализован цикл с задержкой *Sleep*(500) и выводом *PID* на каждой итерации. Число итераций читается из аргумента командной строки, при его отсутствии – из *ITER_NUM*. Если не задан ни один источник, вызывается *ExitProcess*(1). Исходный текст приведён в листинге А.6. Запуски с аргументом 5, с *ITER_NUM*=7 и без параметров показаны на рисунках 2.2, 2.3 и 2.4.

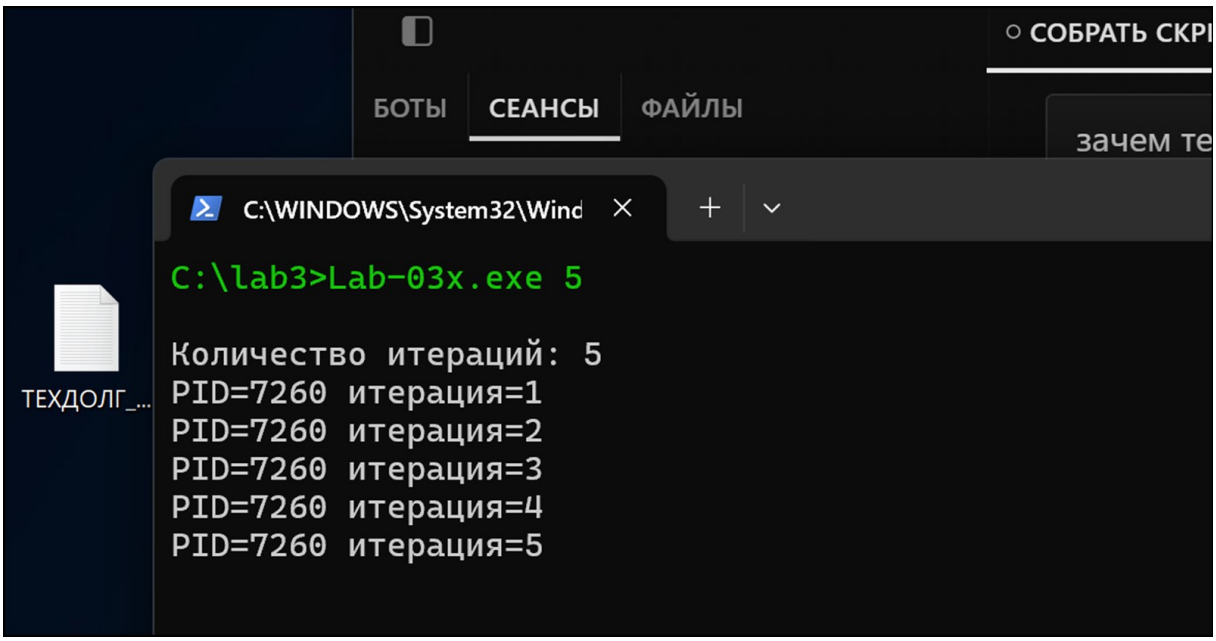


Рисунок 2.2 – *Lab*-03*x.exe* 5: 5 итераций, вывод *PID*

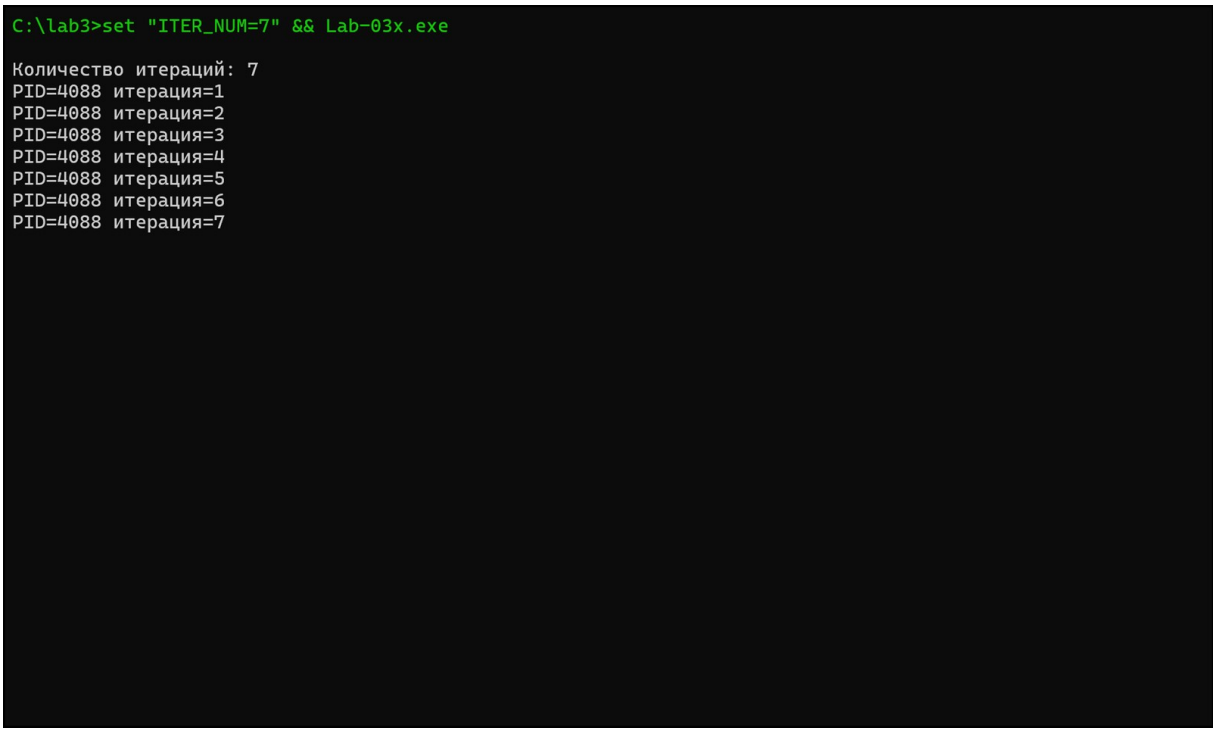


Рисунок 2.3 – *Lab*-03*x.exe* через *set ITER_NUM*=7: 7 итераций

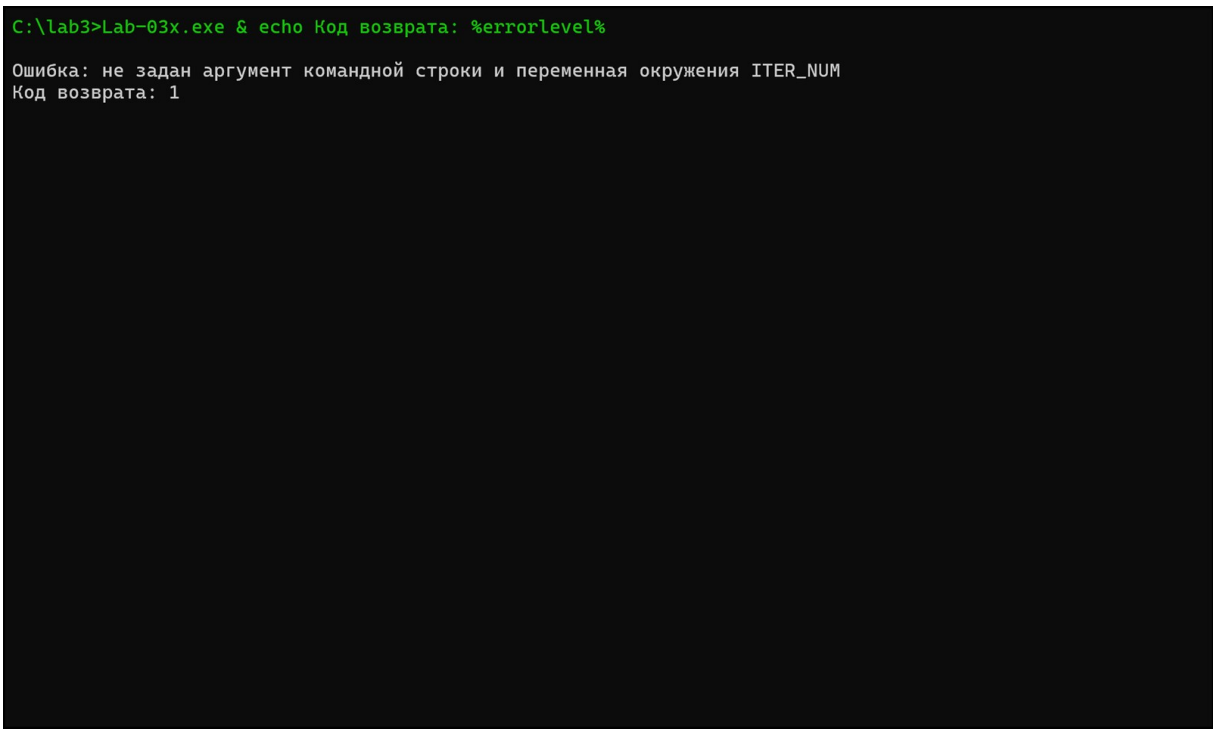


Рисунок 2.4 – *Lab*-03*x.exe* без параметров: ошибка, код возврата 1

Программа *Lab*-03*a* выполняет три вызова *CreateProcess* для запуска *Lab*-03*x*. В первом вызове полный путь и число итераций переданы только через *lpApplicationName*; получена ошибка 2, так как аргумент включён в имя исполняемого файла. Во втором вызове путь и число итераций переданы через *lpCommandLine*, в третьем путь задан в *lpApplicationName*, число итераций – в *lpCommandLine* с ведущим пробелом. Последние два процесса выполнили по пять итераций; родитель дождался завершения и закрыл дескрипторы (рисунок 2.5). Исходный код приведён в листинге А.7.

```
C:\lab3>Lab-03a.exe

Дочерняя программа: C:\lab3\Lab-03x.exe (количество итераций: 5)

[1] lpApplicationName="C:\lab3\Lab-03x.exe 5", lpCommandLine=NULL
    НЕ создан, код ошибки 2

[2] lpApplicationName=NULL, lpCommandLine="C:\lab3\Lab-03x.exe 5"
    процесс создан, PID=544

[3] lpApplicationName="C:\lab3\Lab-03x.exe", lpCommandLine=" 5"
Количество итераций: 5
PID=544 итерация=1
    процесс создан, PID=9084

Родитель ожидает завершения дочерних процессов (2)...
Количество итераций: 5
PID=9084 итерация=1
PID=544 итерация=2
PID=9084 итерация=2
PID=544 итерация=3
PID=9084 итерация=3
PID=9084 итерация=4
PID=544 итерация=4
PID=9084 итерация=5
PID=544 итерация=5
Дочерний процесс 544 завершён (код 0)
Дочерний процесс 9084 завершён (код 0)
Все созданные дочерние процессы завершены, родитель завершает работу.
```

Рисунок 2.5 – *Lab*-03*a.exe* (без аргументов командной строки): [1] ошибка 2, [2] и [3] созданы, по 5 итераций

Для проверки наследования окружения *Lab*-03*b* запущен при *ITER_NUM*=5. Первый дочерний процесс получил это значение из окружения; второй получил аргумент 5. Перед третьим запуском родитель установил *ITER_NUM*=3 через *SetEnvironmentVariableA* и передал *NULL* в *lpCommandLine*. В результате созданы три процесса с числом итераций 5, 5 и 3 соответственно. Исходный текст приведён в листинге А.8, результат – на рисунке 2.6.

```
C:\lab3>set "ITER_NUM=5" && Lab-03b.exe

Дочерняя программа: C:\lab3\Lab-03x.exe
Глобальная ITER_NUM=5, локальная ITER_NUM=3

[1] lpApplicationName="C:\lab3\Lab-03x.exe", lpCommandLine=NULL – глобальная ITER_NUM
    процесс создан, PID=7240

[2] lpApplicationName=NULL, lpCommandLine="C:\lab3\Lab-03x.exe 5"
Количество итераций: 5
PID=7240 итерация=1
    процесс создан, PID=1256

[3] lpApplicationName="C:\lab3\Lab-03x.exe", lpCommandLine=NULL – локальная ITER_NUM=3
    процесс создан, PID=2400

Родитель ожидает завершения дочерних процессов (3)...
Количество итераций: 5
PID=1256 итерация=1
Количество итераций: 3
PID=2400 итерация=1
PID=7240 итерация=2
PID=2400 итерация=2
PID=1256 итерация=2
PID=7240 итерация=3
PID=1256 итерация=3
PID=2400 итерация=3
PID=7240 итерация=4
PID=1256 итерация=4
PID=7240 итерация=5
PID=1256 итерация=5
Дочерний процесс 7240 завершён (код 0)
Дочерний процесс 1256 завершён (код 0)
```

Рисунок 2.6 – Запуск *Lab*-03*b.exe* с *ITER_NUM*=5: глобальная 5, аргумент 5, локальная 3 – все три процесса созданы

Для получения списка процессов в *Lab*-03*c* использованы *CreateToolhelp*32*Snapshot* и *Process*32*First*/*Process*32*Next*. Для каждой записи выводятся имя исполняемого файла, *PID* и *PPID*. Исходный текст приведён в листинге А.9. На рисунке 2.7 показана запись *Lab*-03*c.exe* с *PID* 8336 и *PPID* 7244, соответствующим *cmd.exe*.

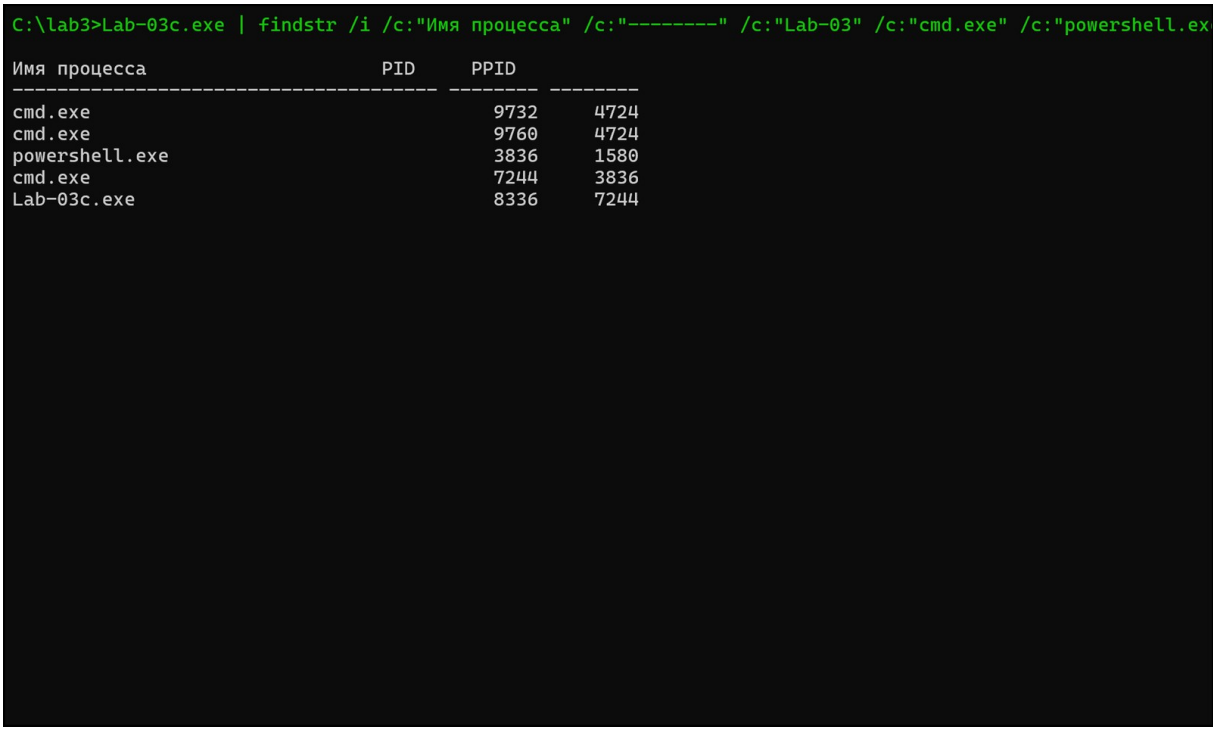


Рисунок 2.7 – *Lab*-03*c.exe*: таблица процессов (имя, *PID*, *PPID*), *Lab*-03*c.exe* (*PID*=8336) из *cmd.exe* (*PID*=7244)

Родительские связи дополнительно проверены средствами наблюдения за процессами. На рисунке 2.8 показаны *Lab*-03*a.exe* и два дочерних *Lab*-03*x.exe*.

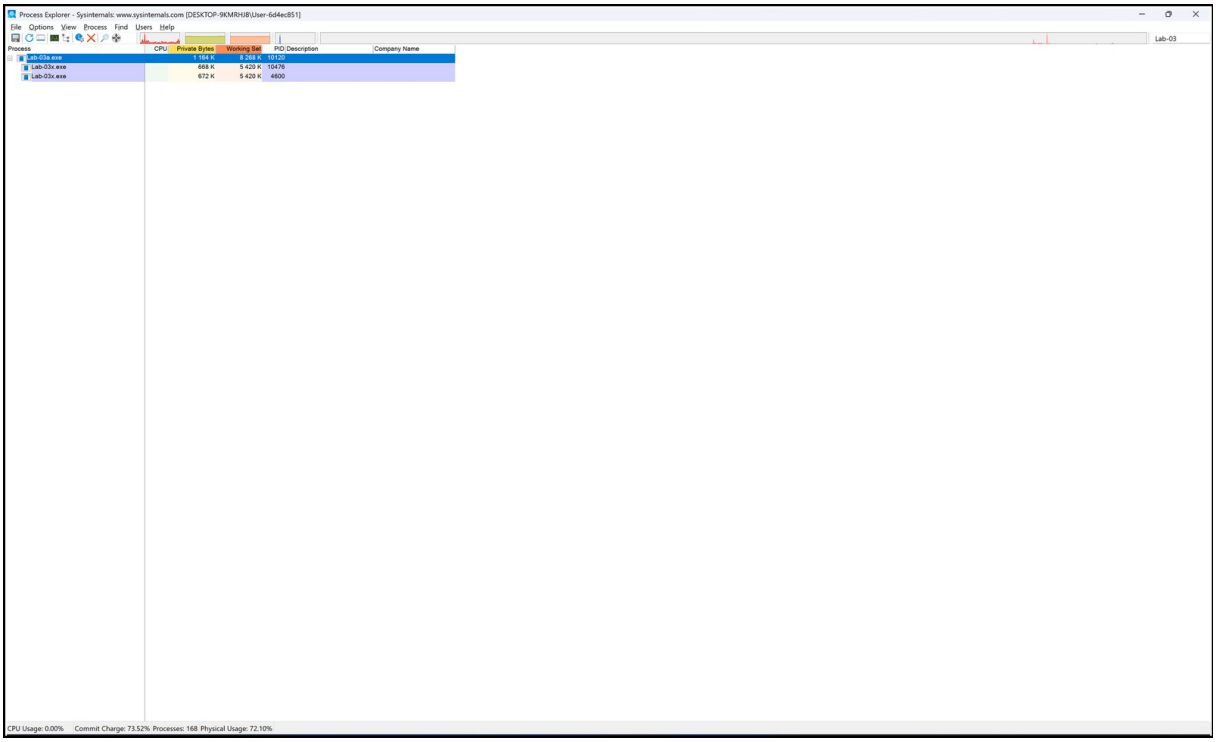


Рисунок 2.8 – Три одновременных процесса: *Lab*-03*a.exe* (*PID* 10120) и два дочерних *Lab*-03*x.exe* (*PID* 10476, 4600)

Команда *tasklist* с фильтром *findstr* показывает *Lab*-03*a* и два *Lab*-03*x* на рисунке 2.9. В Диспетчере задач на рисунке 2.10 отображаются родительский и два дочерних процесса; *PID* относятся к показанному запуску и могут отличаться от консольного примера. В нижней панели *Handles Process Explorer* видны дескрипторы дочерних процессов у *Lab*-03*a* на рисунке 2.11.

```
C:\lab3>tasklist | findstr /i "Lab-03"

Lab-03a.exe                   6188 Console                    1     8 032 КБ
Lab-03x.exe                  11080 Console                    1     5 196 КБ
Lab-03x.exe                  10496 Console                    1     5 200 КБ
```

Рисунок 2.9 – *tasklist* | *findstr* /*i Lab*-03: процессы *Lab*-03*a* и два *Lab*-03*x* с *PID*

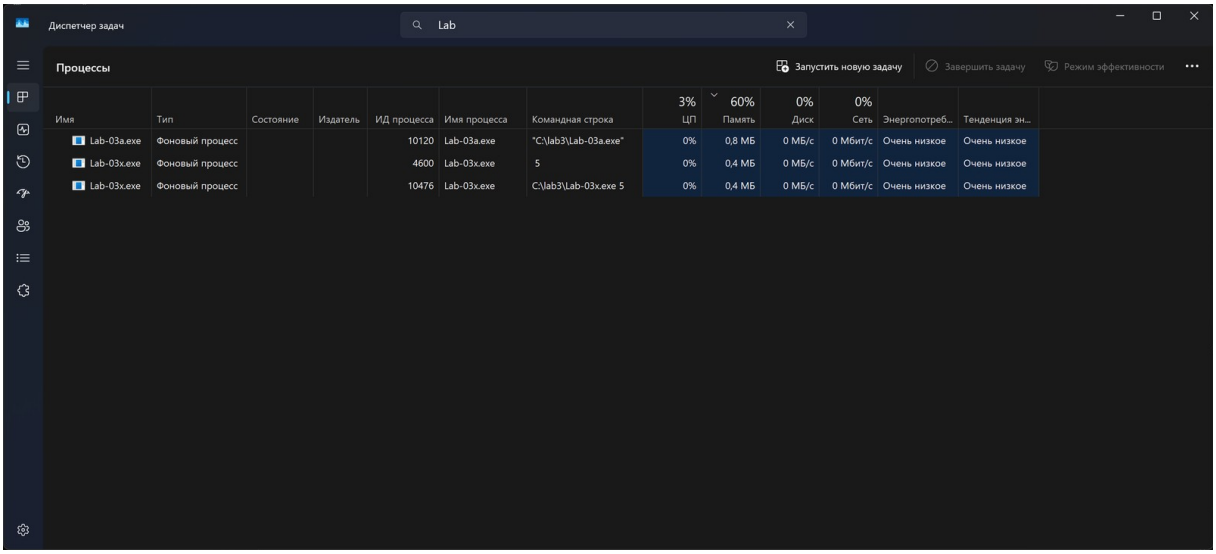


Рисунок 2.10 – Диспетчер задач, вкладка «Подробности», поиск «*Lab*»: *Lab*-03*a.exe* (10120) и два *Lab*-03*x.exe* (4600, 10476)

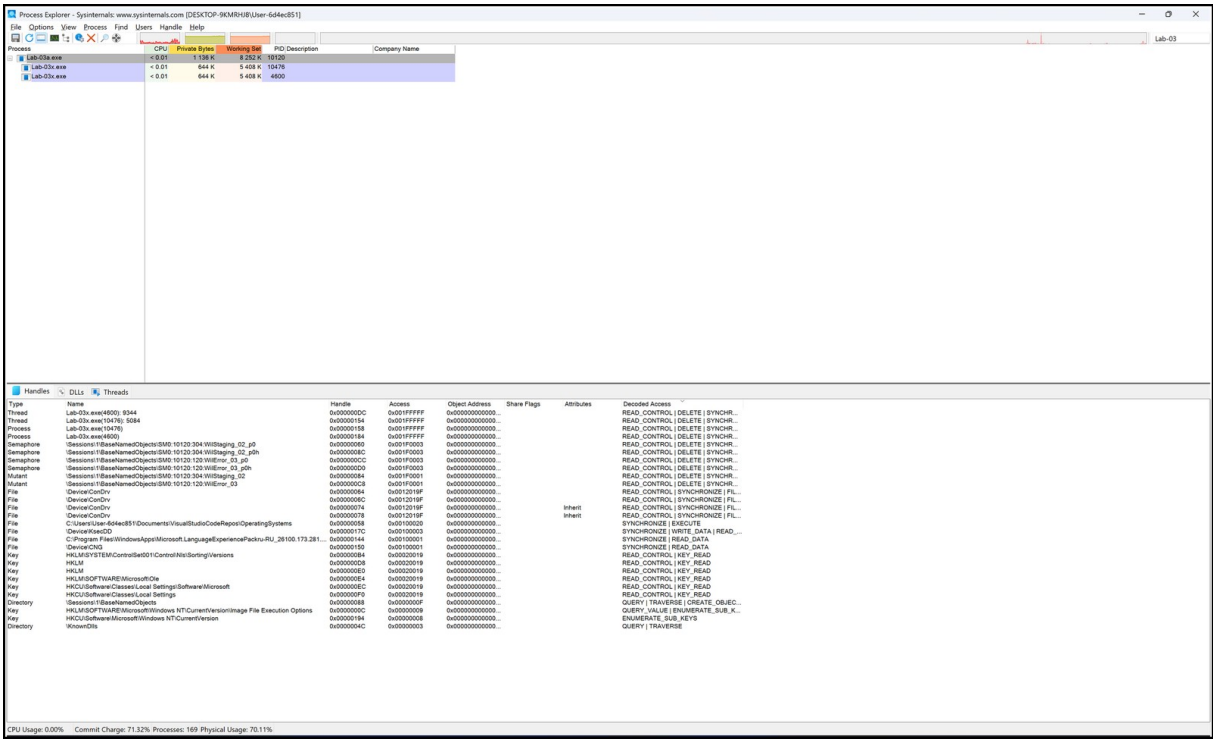


Рисунок 2.11 – *Process Explorer*: нижняя панель *Handles* процесса *Lab*-03*a* – дескрипторы дочерних *Lab*-03*x*

Сопоставление запусков позволило отделить ошибку передачи аргумента в *lpApplicationName* от корректной передачи командной строки. Пример *Lab*-03*b* показал наследование текущего окружения при создании процесса, а *Lab*-03*c* – получение родительских идентификаторов через *Tool Help Library*.

2.3 Уровень 7–8: *IPC* через канал и *STARTUPINFO*

Для параллельного поиска чисел, взаимно простых с *M*, сервер *Lab*-03*d-server* создаёт анонимный канал через *CreatePipe* и запускает *K* экземпляров *Lab*-03*d-client*. Дескриптор записи передаётся через *STARTUPINFO.hStdOutput* с

флагом *STARTF_USESTDHANDLES* и разрешённым наследованием дескрипторов. Каждый клиент обрабатывает свой поддиапазон и передаёт сообщение *CHUNK* с *PID*, количеством и найденными числами. Сервер читает канал, ожидает завершения клиентов и выводит общий результат; время измеряется через *QueryPerformanceCounter*. Исходные тексты приведены в листингах А.10 и А.11. Результаты запусков сервера с тремя и четырьмя процессами показаны на рисунках 2.12 и 2.13.

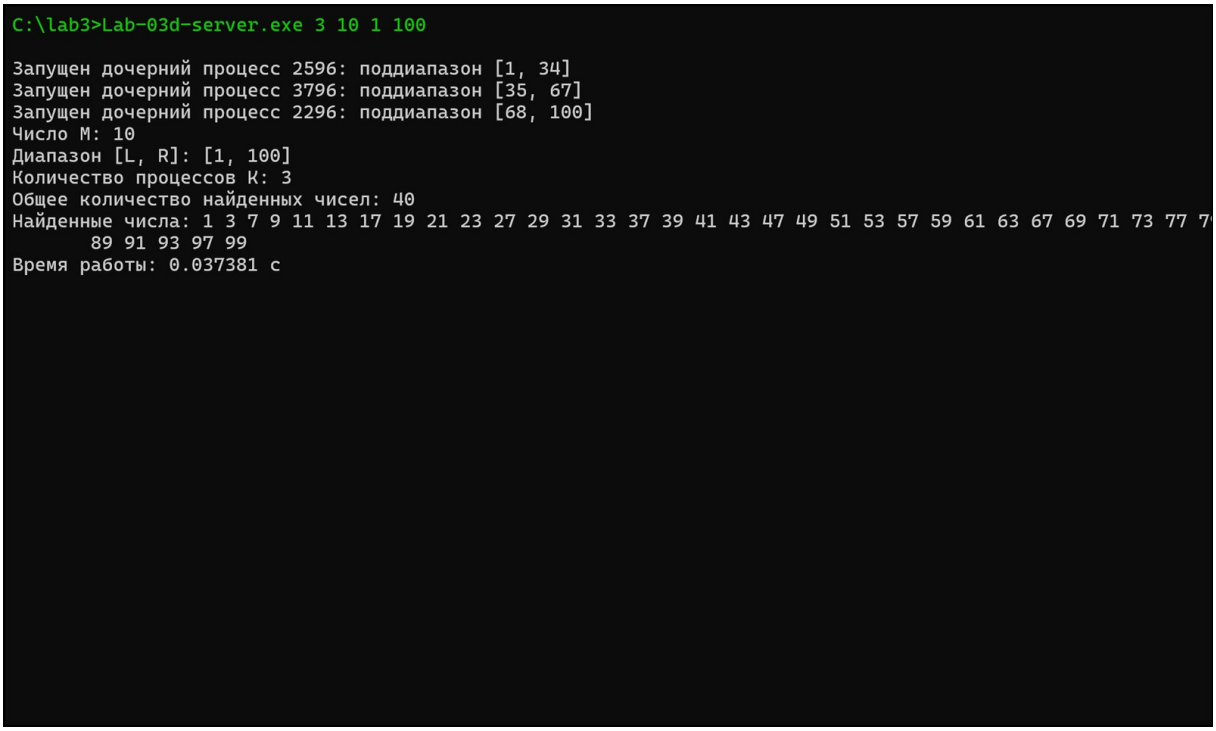


Рисунок 2.12 – *Lab*-03*d-server*.*exe*: *K*=3, *M*=10, диапазон [1, 100], найдено 40 чисел

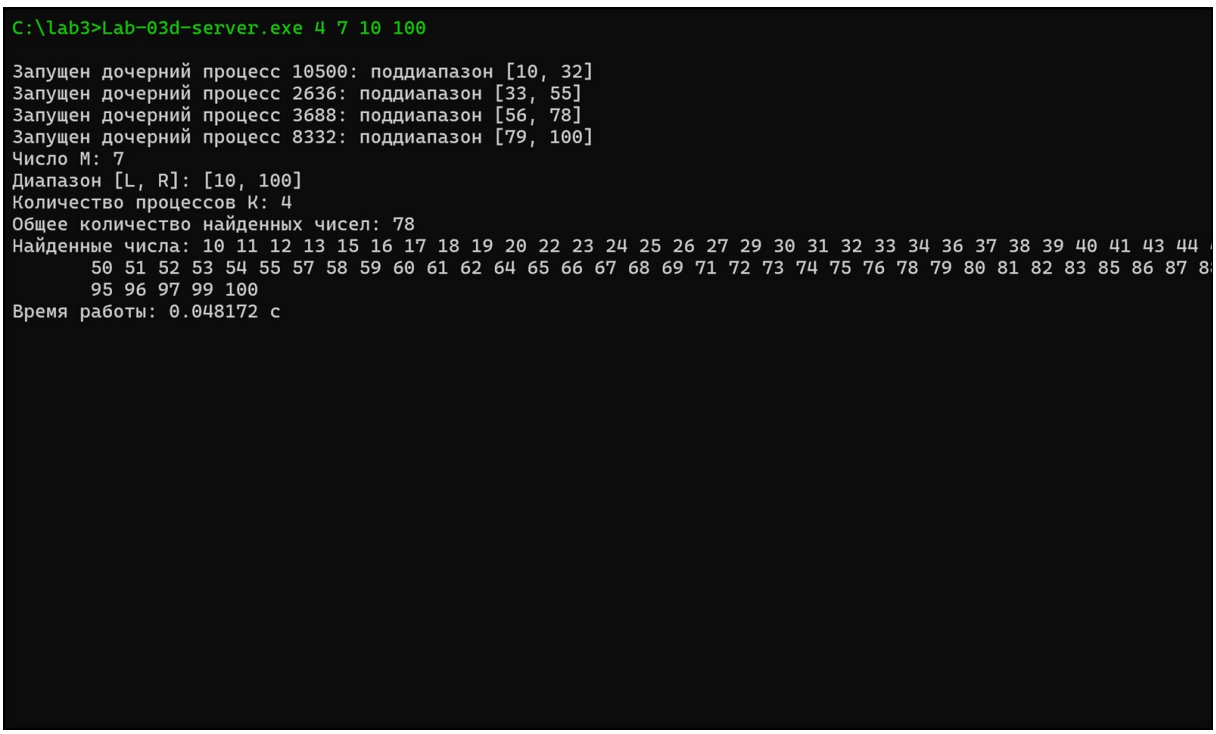


Рисунок 2.13 – *Lab*-03*d-server*.*exe* 4 7 10 100: *K*=4, другой *M* и диапазон

При *K* = 3, *M* = 10 и диапазоне [1, 100] сервер нашёл 40 чисел. Для *K* = 4, *M* = 7 и диапазона [10, 100] получено 78 чисел. Время измерено *QueryPerformanceCounter*; на этих запусках получено 0,037381 и 0,048172 с соответственно.

В нижней панели *Handles Process Explorer* виден объект *\Device\NamedPipe*, связанный с каналом, и дескрипторы дочерних процессов *Lab-03d-client*. Снимок показывает канал в списке дескрипторов (рисунок 2.14).

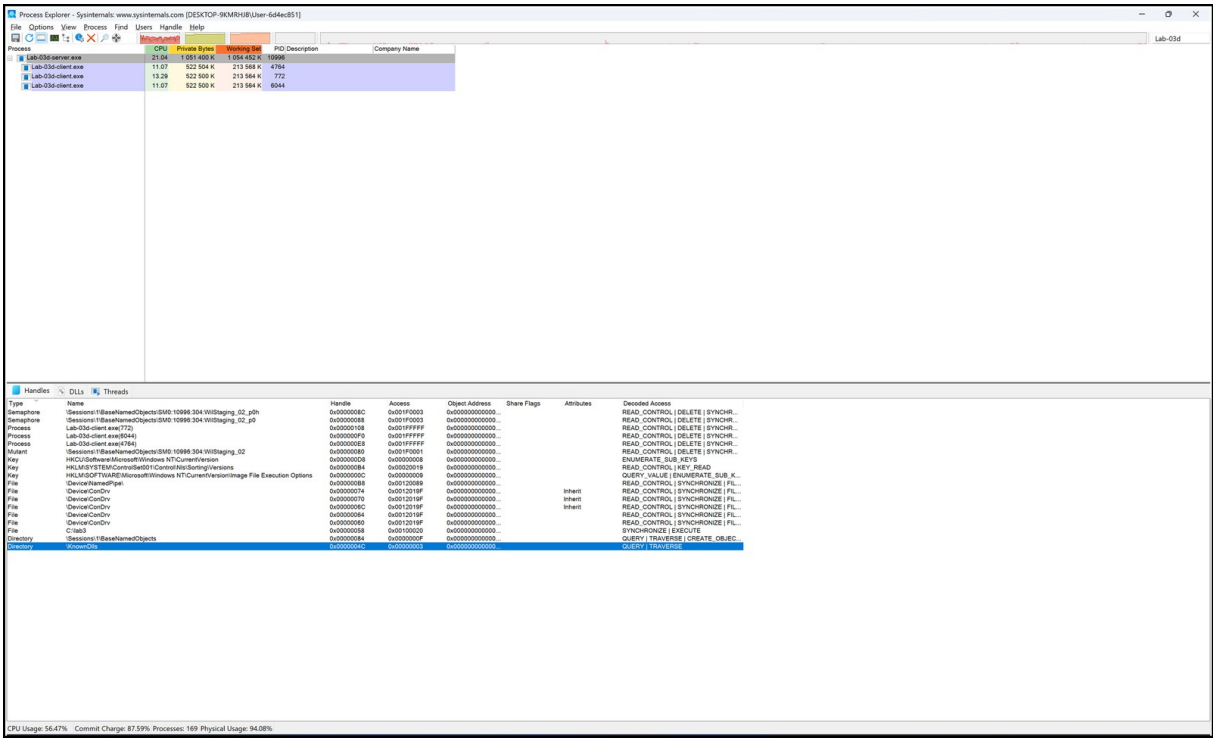


Рисунок 2.14 – *Process Explorer*: канал *\Device\NamedPipe* и дескрипторы дочерних клиентов

Разделение диапазона позволило распределить проверку чисел между клиентами. Наследуемый дескриптор связал стандартный вывод клиентов с каналом сервера, а чтение результатов и ожидание процессов обеспечили завершение родителя после выполнения дочерних вычислений.

2.4 Уровень 9–10: передача через *STARTUPINFO*

Для передачи фразы «*Hello, my dear child*!» родитель записывает её в *STARTUPINFO.lpTitle* и создаёт дочерний процесс с *CREATE_NEW_CONSOLE*. Дочерняя программа считывает заголовок консоли через *GetConsoleTitleA* и выводит полученную строку. Для передачи фразы используются *lpTitle* и создание новой консоли, а не аргументы командной строки, переменные окружения или перенаправление потоков. Исходные тексты родителя и дочерней программы приведены в листингах А.12 и А.13. Полученная дочерним процессом строка показана на рисунке 2.15. Состав исполняемых файлов приведён на рисунке 2.16.

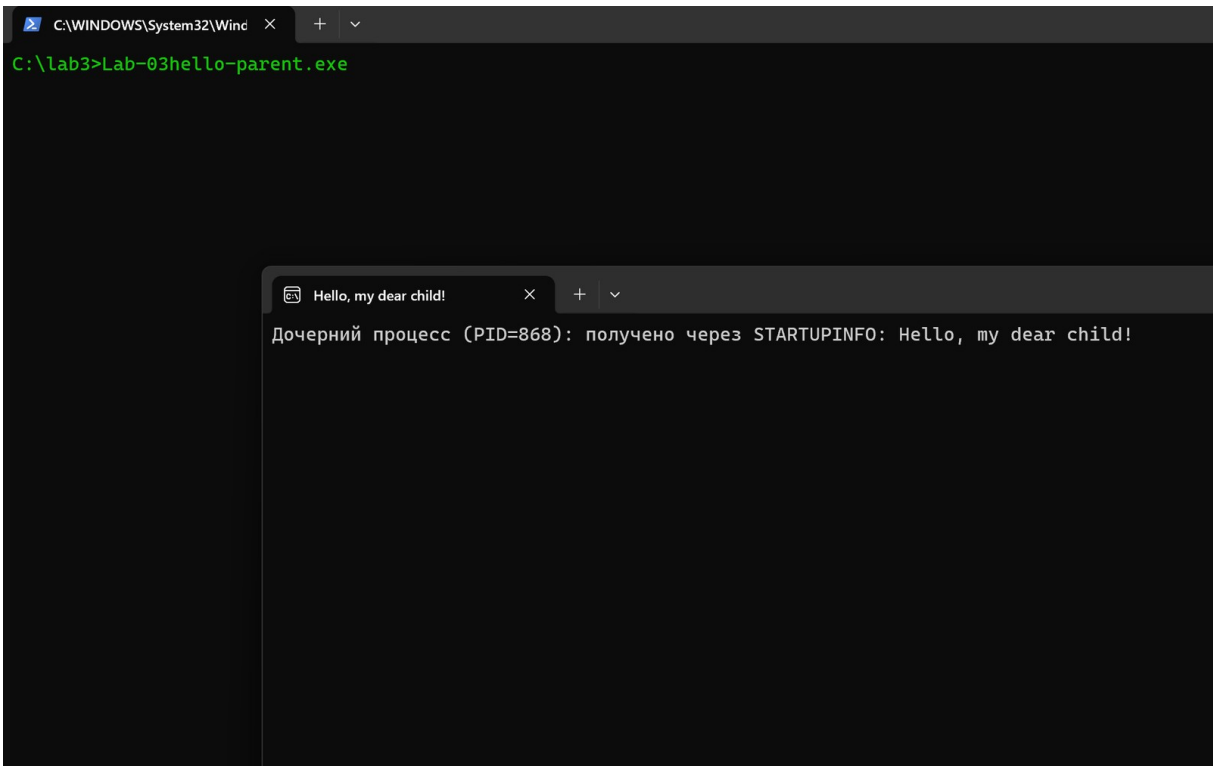


Рисунок 2.15 – *Lab*-03*hello-parent.exe*: родитель создаёт дочерний с *CREATE_NEW_CONSOLE* и *lpTitle*; дочернее окно показывает фразу

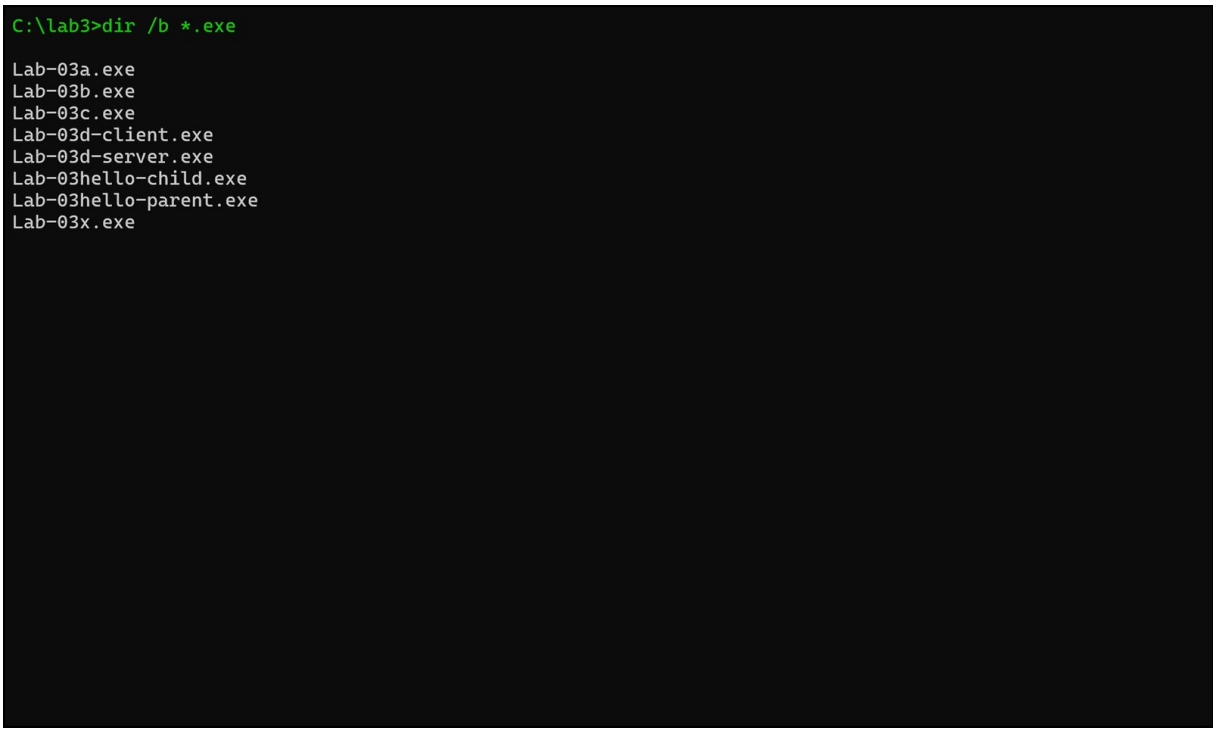


Рисунок 2.16 – Список собранных *exe* в *C*:*lab*3

Дочерний процесс вывел фразу, заданную родителем в заголовке новой консоли. В этом примере *STARTUPINFO* определяет начальные параметры консоли, а *GetConsoleTitleA* позволяет прочитать переданное значение.

2.5 Вывод по разделу

В *Windows* запуск исполняемого файла и создание процесса выполняются вызовом *CreateProcess*. Примеры показали различие между именем программы и командной строкой, наследование окружения и дескрипторов, получение списка процессов и ожидание завершения. В задачах обмена данными *STARTUPINFO* использована для настройки стандартного вывода клиента и заголовка новой консоли.

Заключение

В ходе работы исследованы создание и завершение процессов, передача параметров, получение сведений о процессах и обмен данными между родителем и дочерними программами в *Linux* и *Windows*.

В *Linux* использованы *fork*, *execl* и *waitpid*. Для обмена результатами применён анонимный канал с перенаправлением *stdout* через *dup*2. Пример обработки *SIGINT* показал, что реакция процесса на этот сигнал может быть изменена обработчиком.

В *Windows* рассмотрены варианты вызова *CreateProcess* и наследование *ITER_NUM*. Параллельный поиск чисел выполнен с передачей дескриптора канала через *STARTUPINFO*, а отдельная пара программ продемонстрировала передачу строки через заголовок консоли.

Ошибка 2 в первом вызове *Lab*-03*a* связана не с неработоспособностью *lpApplicationName*, а с включением аргумента в строку имени файла. Полученные результаты подтверждают необходимость раздельно учитывать путь к программе, командную строку, окружение и наследуемые дескрипторы при создании процесса.

ПРИЛОЖЕНИЕ А

Исходные тексты программ

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

static int should_pause(void)
{
    return getenv("PAUSE_ON_EXIT") != NULL;
}

static void pause_before_exit(void)
{
    printf("PID=%d: press Enter to exit...\n", getpid());
    fflush(stdout);
    getchar();
}

int main(int argc, char *argv[])
{
    int iterations;
    int pause = should_pause();

    if (argc > 1 && strcmp(argv[1], "--pause") != 0) {
        iterations = atoi(argv[1]);
    } else {
        const char *env = getenv("ITER_NUM");
        if (!env) {
            fprintf(stderr, "Error: no argument or ITER_NUM env
var\n");
            return 1;
        }
        iterations = atoi(env);
    }

    printf("Iterations: %d\n", iterations);

    for (int i = 1; i <= iterations; i++) {
        printf("PID=%d iteration=%d\n", getpid(), i);
        fflush(stdout);
        usleep(500000);
    }

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.1 – *Lab*-03*x.c* (*Linux*)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define FIRST_ITER  "5"
#define SECOND_ITER "8"

int main(void)
{
    pid_t pid1, pid2;

    pid1 = fork();
    if (pid1 < 0) {
        perror("fork (1)");
        return 1;
    }
    if (pid1 == 0) {
        execl("./Lab-03x", "Lab-03x", FIRST_ITER, (char *)NULL);
        perror("execl (1)");
        _exit(1);
    }

    setenv("ITER_NUM", SECOND_ITER, 1);
    pid2 = fork();
    if (pid2 < 0) {
        perror("fork (2)");
        return 1;
    }
    if (pid2 == 0) {
        execl("./Lab-03x", "Lab-03x", (char *)NULL);
        perror("execl (2)");
        _exit(1);
    }

    printf("Parent PID=%d waiting for children...\n", getpid());
    printf("  Child 1: PID=%d (5 iterations)\n", pid1);
    printf("  Child 2: PID=%d (8 iterations)\n", pid2);
    printf("  >>> Run 'ps -ef | grep Lab-03x' in another terminal
<<<\n");
    fflush(stdout);

    int status;
    waitpid(pid1, &status, 0);
    printf("Child %d exited (code %d)\n", pid1, WEXITSTATUS(status));
    waitpid(pid2, &status, 0);
    printf("Child %d exited (code %d)\n", pid2, WEXITSTATUS(status));

    printf("All children done, parent exiting.\n");
    return 0;
}
```

Листинг А.2 – *Lab*-03*a.c* (*Linux*)

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>

static int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

static int should_pause(void)
{
    return getenv("PAUSE_ON_EXIT") != NULL;
}

static void pause_before_exit(void)
{
    printf("PID=%d: press Enter to exit...\n", getpid());
    fflush(stdout);
    getchar();
}

int main(int argc, char *argv[])
{
    if (argc != 4) {
        fprintf(stderr, "Usage: %s <M> <L> <R>\n", argv[0]);
        return 1;
    }

    int m = atoi(argv[1]);
    int l = atoi(argv[2]);
    int r = atoi(argv[3]);

    if (m <= 0 || l < 1 || l > r) {
        fprintf(stderr, "Error: M > 0, L >= 1, L <= R required\n");
        return 1;
    }

    int *found = malloc((size_t)(r - l + 1) * sizeof(int));
    if (!found) {
        perror("malloc");
        return 1;
    }
    int count = 0;

    for (int n = l; n <= r; n++) {
        if (gcd(n, m) == 1) {
            found[count++] = n;
```

```
        }
    }

    printf("CHUNK %d %d %d", getpid(), count, 0);
    for (int i = 0; i < count; i++) {
        printf(" %d", found[i]);
    }
    printf("\n");
    fflush(stdout);

    free(found);

    if (should_pause()) pause_before_exit();
    return 0;
}
```

Листинг А.3 – *Lab*-03*d-client.c* (*Linux*)

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define MAX_NUMBERS 100000

static double now_seconds(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

int main(int argc, char *argv[])
{
    if (argc != 5) {
        fprintf(stderr, "Usage: %s <K> <M> <L> <R>\n", argv[0]);
        return 1;
    }

    int k = atoi(argv[1]);
    int m = atoi(argv[2]);
    int l = atoi(argv[3]);
    int r = atoi(argv[4]);

    if (k < 1 || m <= 0 || l < 1 || l > r) {
        fprintf(stderr, "Error: K >= 1, M > 0, L >= 1, L <= R
required\n");
        return 1;
    }

    double t0 = now_seconds();

    int fd[2];
    if (pipe(fd) < 0) {
        perror("pipe");
        return 1;
    }

    int total = r - l + 1;
    int chunk = total / k;
    int remainder = total % k;

    pid_t *pids = malloc((size_t)k * sizeof(pid_t));
    if (!pids) {
        perror("malloc");
        return 1;
    }

    int sub_l = l;
```

```
    for (int i = 0; i < k; i++) {
        int sub_r = sub_l + chunk - 1 + (i < remainder ? 1 : 0);
        if (i == k - 1) sub_r = r;

        pid_t pid = fork();
        if (pid < 0) {
            perror("fork");
            return 1;
        }
        if (pid == 0) {
            close(fd[0]);
            dup2(fd[1], STDOUT_FILENO);
            close(fd[1]);

            char ml[32], ll[32], rl[32];
            snprintf(ml, sizeof(ml), "%d", m);
            snprintf(ll, sizeof(ll), "%d", sub_l);
            snprintf(rl, sizeof(rl), "%d", sub_r);
            execl("./Lab-03d-client", "Lab-03d-client", ml, ll, rl,
(char *)NULL);
            perror("execl");
            _exit(1);
        }
        pids[i] = pid;
        sub_l = sub_r + 1;
    }

    close(fd[1]);
    FILE *in = fdopen(fd[0], "r");
    if (!in) {
        perror("fdopen");
        return 1;
    }

    int *all = malloc(MAX_NUMBERS * sizeof(int));
    if (!all) {
        perror("malloc");
        return 1;
    }
    int total_found = 0;

    char line[1 << 16];
    while (fgets(line, sizeof(line), in) != NULL) {
        if (line[0] != 'C') continue;
        int pid_val, count, reserved;
        if (sscanf(line, "CHUNK %d %d %d", &pid_val, &count,
&reserved) != 3) continue;
        char *p = line;
        for (int i = 0; i < 4; i++) {
            p = strchr(p + 1, ' ');
            if (!p) break;
        }
        if (!p) continue;
```

```
        for (int i = 0; i < count && total_found < MAX_NUMBERS; i++)
{
            all[total_found++] = (int)strtol(p, &p, 10);
        }
    }
    fclose(in);

    for (int i = 0; i < k; i++) {
        int status;
        waitpid(pids[i], &status, 0);
    }

    double t1 = now_seconds();

    printf("M: %d\n", m);
    printf("Range [L, R]: [%d, %d]\n", l, r);
    printf("Processes K: %d\n", k);
    printf("Total found: %d\n", total_found);
    printf("Numbers:");
    for (int i = 0; i < total_found; i++) {
        printf(" %d", all[i]);
    }
    printf("\n");
    printf("Time: %.6f s\n", t1 - t0);

    free(all);
    free(pids);
    return 0;
}
```

Листинг А.4 – *Lab*-03*d-server.c* (*Linux*)

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>

static volatile sig_atomic_t sigint_count = 0;

static void on_sigint(int sig)
{
    (void)sig;
    sigint_count++;
    signal(SIGINT, on_sigint);
    const char msg[] = "\n[!] SIGINT (Ctrl+C) caught - continuing\n";
    write(STDOUT_FILENO, msg, sizeof(msg) - 1);
}

int main(void)
{
    signal(SIGINT, on_sigint);

    printf("Lab-03sig started (PID=%d)\n", getpid());
    printf("Ctrl+C cannot stop this application.\n");
    fflush(stdout);

    int second = 0;
    for (;;) {
        sleep(1);
        second++;
        printf("Running... %d s (SIGINT caught: %d)\n", second,
(int)sigint_count);
        fflush(stdout);
    }

    return 0;
}
```

Листинг А.5 – *Lab*-03*sig.c* (*Linux*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "console_ru.h"

/* Пауза перед выходом нужна только для съёмки кадров, где окно
должно
   оставаться на экране; на поведение по методичке она не влияет. */
static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(int argc, char *argv[])
{
    int iterations;
    int pause = should_pause();

    /* Количество итераций: аргумент командной строки, иначе
ITER_NUM,
       иначе – завершение с ошибкой через ExitProcess. */
    if (argc > 1) {
        iterations = atoi(argv[1]);
    } else {
        char buf[32];
        DWORD n = GetEnvironmentVariableA("ITER_NUM", buf,
sizeof(buf));
        if (n == 0 || n >= sizeof(buf)) {
            fprintf(stderr, "Ошибка: не задан аргумент командной
строки и переменная окружения ITER_NUM\n");
            ExitProcess(1);
        }
        iterations = atoi(buf);
    }

    printf("Количество итераций: %d\n", iterations);

    for (int i = 1; i <= iterations; i++) {
        printf("PID=%lu итерация=%d\n", (unsigned
long)GetCurrentProcessId(), i);
        fflush(stdout);
        Sleep(500);
    }
```

```
    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.6 – *Lab*-03*x.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "console_ru.h"

#define DEFAULT_ITER "5"

static void child_path(char *out, size_t cap)
{
    char self[MAX_PATH];
    GetModuleFileNameA(NULL, self, MAX_PATH);
    char *slash = strrchr(self, '\\');
    if (slash != NULL) {
        *(slash + 1) = '\0';
    } else {
        self[0] = '\0';
    }
    snprintf(out, cap, "%sLab-03x.exe", self);
}

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(void)
{
    const char *iter = DEFAULT_ITER;
    int pause = should_pause();

    char child[MAX_PATH];
    child_path(child, sizeof(child));
    printf("Дочерняя программа: %s (количество итераций: %s)\n\n",
child, iter);

    PROCESS_INFORMATION pi[3];
    BOOL ok[3] = {0};
    STARTUPINFOA si = {0};
    si.cb = sizeof(si);

    /* Процесс 1: только lpApplicationName, количество итераций тоже
в нём –
       так задумано по методичке, ожидается ошибка 2. */
```

```
    {
        char app[MAX_PATH + 16];
        snprintf(app, sizeof(app), "%s %s", child, iter);
        printf("[1] lpApplicationName=\"%s\", lpCommandLine=NULL\n",
app);
        ok[0] = CreateProcessA(app, NULL, NULL, NULL, FALSE, 0, NULL,
NULL, &si, &pi[0]);
        if (ok[0]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[0].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    /* Процесс 2: только lpCommandLine. */
    {
        char cmd[MAX_PATH + 16];
        snprintf(cmd, sizeof(cmd), "%s %s", child, iter);
        printf("[2] lpApplicationName=NULL, lpCommandLine=\"%s\"\n",
cmd);
        ok[1] = CreateProcessA(NULL, cmd, NULL, NULL, FALSE, 0, NULL,
NULL, &si, &pi[1]);
        if (ok[1]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[1].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    /* Процесс 3: имя в lpApplicationName, итерации в lpCommandLine.
       Пробел в начале строки обязателен – иначе имя процесса уедет в
argv[0]. */
    {
        char cmd[MAX_PATH + 16];
        snprintf(cmd, sizeof(cmd), " %s", iter);
        printf("[3] lpApplicationName=\"%s\",
lpCommandLine=\"%s\"\n", child, cmd);
        ok[2] = CreateProcessA(child, cmd, NULL, NULL, FALSE, 0,
NULL, NULL, &si, &pi[2]);
        if (ok[2]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[2].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    /* Родитель ждёт завершения всех дочерних процессов (они работают
       одновременно) и только потом завершается сам. */
    HANDLE h[3];
    DWORD count = 0;
    for (int i = 0; i < 3; i++) {
        if (ok[i]) h[count++] = pi[i].hProcess;
    }

    printf("Родитель ожидает завершения дочерних процессов (%lu)...
\n", count);
```

```
    if (count > 0) WaitForMultipleObjects(count, h, TRUE, INFINITE);

    for (int i = 0; i < 3; i++) {
        if (ok[i]) {
            DWORD code = 0;
            GetExitCodeProcess(pi[i].hProcess, &code);
            printf("Дочерний процесс %lu завершён (код %lu)\n",
                   (unsigned long)pi[i].dwProcessId, (unsigned
long)code);
            CloseHandle(pi[i].hProcess);
            CloseHandle(pi[i].hThread);
        }
    }

    printf("Все созданные дочерние процессы завершены, родитель
завершает работу.\n");
    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.7 – *Lab*-03*a.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "console_ru.h"

#define DEFAULT_ITER "5"
#define LOCAL_ITER "3"

static void child_path(char *out, size_t cap)
{
    char self[MAX_PATH];
    GetModuleFileNameA(NULL, self, MAX_PATH);
    char *slash = strrchr(self, '\\');
    if (slash != NULL) *(slash + 1) = '\0';
    else self[0] = '\0';
    snprintf(out, cap, "%sLab-03x.exe", self);
}

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(void)
{
    const char *iter = DEFAULT_ITER;
    int pause = should_pause();

    const char *global_iter = "(не задана)";
    char gbuf[32];
    DWORD glen = GetEnvironmentVariableA("ITER_NUM", gbuf,
sizeof(gbuf));
    if (glen > 0 && glen < sizeof(gbuf)) global_iter = gbuf;

    char child[MAX_PATH];
    child_path(child, sizeof(child));
    printf("Дочерняя программа: %s\n", child);
    printf("Глобальная ITER_NUM=%s, локальная ITER_NUM=%s\n\n",
global_iter, LOCAL_ITER);

    PROCESS_INFORMATION pi[3] = {0};
    BOOL ok[3] = {0};
    STARTUPINFOA si = {0};
```

```
    si.cb = sizeof(si);

    /* Процесс 1: только имя файла – итерации из глобальной ITER_NUM.
*/
    {
        printf("[1] lpApplicationName=\"%s\", lpCommandLine=NULL -
глобальная ITER_NUM\n", child);
        ok[0] = CreateProcessA(child, NULL, NULL, NULL, FALSE, 0,
NULL, NULL, &si, &pi[0]);
        if (ok[0]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[0].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    /* Процесс 2: итерации передаются аргументом в lpCommandLine. */
    {
        char cmd[MAX_PATH + 16];
        snprintf(cmd, sizeof(cmd), "%s %s", child, iter);
        printf("[2] lpApplicationName=NULL, lpCommandLine=\"%s\"\n",
cmd);
        ok[1] = CreateProcessA(NULL, cmd, NULL, NULL, FALSE, 0, NULL,
NULL, &si, &pi[1]);
        if (ok[1]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[1].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    /* Процесс 3: второй параметр NULL, перед вызовом задаётся
локальная
       ITER_NUM – значение отличается от глобального. */
    {
        SetEnvironmentVariableA("ITER_NUM", LOCAL_ITER);
        printf("[3] lpApplicationName=\"%s\", lpCommandLine=NULL -
локальная ITER_NUM=%s\n", child, LOCAL_ITER);
        ok[2] = CreateProcessA(child, NULL, NULL, NULL, FALSE, 0,
NULL, NULL, &si, &pi[2]);
        if (ok[2]) printf("    процесс создан, PID=%lu\n\n",
(unsigned long)pi[2].dwProcessId);
        else printf("    НЕ создан, код ошибки %lu\n\n",
GetLastError());
    }

    HANDLE h[3];
    DWORD count = 0;
    for (int i = 0; i < 3; i++) if (ok[i]) h[count++] =
pi[i].hProcess;

    printf("Родитель ожидает завершения дочерних процессов (%lu)...
\n", count);
    if (count > 0) WaitForMultipleObjects(count, h, TRUE, INFINITE);
```

```
    for (int i = 0; i < 3; i++) {
        if (ok[i]) {
            DWORD code = 0;
            GetExitCodeProcess(pi[i].hProcess, &code);
            printf("Дочерний процесс %lu завершён (код %lu)\n",
                   (unsigned long)pi[i].dwProcessId, (unsigned
long)code);
            CloseHandle(pi[i].hProcess);
            CloseHandle(pi[i].hThread);
        }
    }

    printf("Все созданные дочерние процессы завершены, родитель
завершает работу.\n");
    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.8 – *Lab*-03*b.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <tlhelp32.h>

#include "console_ru.h"

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(void)
{
    int pause = should_pause();

    HANDLE snap = CreateToolhelp32Snapshot(TH32CS_SNAPPROCESS, 0);
    if (snap == INVALID_HANDLE_VALUE) {
        fprintf(stderr, "Ошибка: CreateToolhelp32Snapshot (%lu)\n",
GetLastError());
        return 1;
    }

    PROCESSENTRY32 pe;
    pe.dwSize = sizeof(pe);

    printf("%-38s %8s %8s\n", "Имя процесса", "PID", "PPID");
    printf("%-38s %8s %8s\n",
"--------------------------------------", "--------", "--------");

    if (Process32First(snap, &pe)) {
        do {
            printf("%-38s %8lu %8lu\n",
                   pe.szExeFile, (unsigned long)pe.th32ProcessID,
                   (unsigned long)pe.th32ParentProcessID);
        } while (Process32Next(snap, &pe));
    }

    CloseHandle(snap);

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг A.9 – *Lab*-03*c.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "console_ru.h"

static int gcd(int a, int b)
{
    while (b != 0) {
        int t = b;
        b = a % b;
        a = t;
    }
    return a;
}

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(int argc, char *argv[])
{
    int pause = should_pause();

    /* Структура вызова: Lab-03d-client <Число M> <Нижний порог>
<Верхний порог> */
    if (argc != 4) {
        fprintf(stderr, "Использование: %s <M> <L> <R>\n", argv[0]);
        return 1;
    }

    int m = atoi(argv[1]);
    int l = atoi(argv[2]);
    int r = atoi(argv[3]);

    if (m <= 0 || l < 1 || l > r) {
        fprintf(stderr, "Ошибка: требуется M > 0, L >= 1, L <= R\n");
        return 1;
    }

    int *found = malloc((size_t)(r - l + 1) * sizeof(int));
    if (!found) {
        fprintf(stderr, "Ошибка: не удалось выделить память\n");
```

```
        return 1;
    }
    int count = 0;

    /* Числа, взаимно простые с M: НОД(n, M) == 1. */
    for (int n = l; n <= r; n++) {
        if (gcd(n, m) == 1) {
            found[count++] = n;
        }
    }

    /* Результат уходит в анонимный канал, который родитель передал
       через STARTUPINFO (hStdOutput). */
    printf("CHUNK %lu %d %d", (unsigned long)GetCurrentProcessId(),
count, 0);
    for (int i = 0; i < count; i++) {
        printf(" %d", found[i]);
    }
    printf("\n");
    fflush(stdout);

    free(found);

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.10 – *Lab*-03*d-client.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#include "console_ru.h"

#define MAX_NUMBERS 1000000

static double now_seconds(void)
{
    LARGE_INTEGER freq, cnt;
    QueryPerformanceFrequency(&freq);
    QueryPerformanceCounter(&cnt);
    return (double)cnt.QuadPart / (double)freq.QuadPart;
}

static void client_path(char *out, size_t cap)
{
    char self[MAX_PATH];
    GetModuleFileNameA(NULL, self, MAX_PATH);
    char *slash = strrchr(self, '\\');
    if (slash != NULL) *(slash + 1) = '\0';
    else self[0] = '\0';
    snprintf(out, cap, "%sLab-03d-client.exe", self);
}

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(int argc, char *argv[])
{
    int pause = should_pause();

    /* Структура вызова: Lab-03d-server <K> <Число M> <Нижний порог>
<Верхний порог> */
    if (argc != 5) {
        fprintf(stderr, "Использование: %s <K> <M> <L> <R>\n",
argv[0]);
        return 1;
    }

    int k = atoi(argv[1]);
```

```
    int m = atoi(argv[2]);
    int l = atoi(argv[3]);
    int r = atoi(argv[4]);

    if (k < 1 || m <= 0 || l < 1 || l > r) {
        fprintf(stderr, "Ошибка: требуется K >= 1, M > 0, L >= 1, L
<= R\n");
        return 1;
    }

    char client[MAX_PATH];
    client_path(client, sizeof(client));

    double t0 = now_seconds();

    /* Анонимный канал: дескриптор записи наследуется дочерними
процессами. */
    SECURITY_ATTRIBUTES sa = {0};
    sa.nLength = sizeof(sa);
    sa.bInheritHandle = TRUE;

    HANDLE hRead, hWrite;
    if (!CreatePipe(&hRead, &hWrite, &sa, 0)) {
        fprintf(stderr, "Ошибка: CreatePipe (%lu)\n",
GetLastError());
        return 1;
    }
    SetHandleInformation(hRead, HANDLE_FLAG_INHERIT, 0);

    HANDLE *procs = malloc((size_t)k * sizeof(HANDLE));
    if (!procs) {
        fprintf(stderr, "Ошибка: не удалось выделить память\n");
        return 1;
    }
    int created = 0;

    int total = r - l + 1;
    int chunk = total / k;
    int remainder = total % k;
    int subL = l;

    /* Диапазон [L, R] делится на K равных частей, каждый дочерний
процесс
       обрабатывает свой поддиапазон. */
    for (int i = 0; i < k; i++) {
        int subR = subL + chunk - 1 + (i < remainder ? 1 : 0);
        if (i == k - 1) subR = r;

        STARTUPINFOA si = {0};
        si.cb = sizeof(si);
        si.dwFlags = STARTF_USESTDHANDLES;
        si.hStdInput = GetStdHandle(STD_INPUT_HANDLE);
        si.hStdOutput = hWrite;
```

```
        si.hStdError = hWrite;

        PROCESS_INFORMATION pi = {0};

        char cmd[MAX_PATH + 64];
        snprintf(cmd, sizeof(cmd), "\"%s\" %d %d %d", client, m,
subL, subR);

        if (!CreateProcessA(NULL, cmd, NULL, NULL, TRUE, 0, NULL,
NULL, &si, &pi)) {
            fprintf(stderr, "Ошибка: CreateProcess для [%d, %d] (%lu)
\n", subL, subR, GetLastError());
        } else {
            CloseHandle(pi.hThread);
            procs[created++] = pi.hProcess;
            printf("Запущен дочерний процесс %lu: поддиапазон [%d,
%d]\n",
                   (unsigned long)pi.dwProcessId, subL, subR);
        }
        subL = subR + 1;
    }

    CloseHandle(hWrite);

    size_t cap = 1 << 16;
    char *data = malloc(cap);
    if (!data) {
        fprintf(stderr, "Ошибка: не удалось выделить память\n");
        return 1;
    }
    size_t used = 0;

    /* Читаем канал до EOF: EOF наступает, когда все дочерние
процессы
       закрыли унаследованный дескриптор записи. */
    for (;;) {
        DWORD got = 0;
        if (used + 4096 >= cap) {
            cap *= 2;
            char *bigger = realloc(data, cap);
            if (!bigger) break;
            data = bigger;
        }
        if (!ReadFile(hRead, data + used, (DWORD)(cap - used - 1),
&got, NULL) || got == 0) {
            break;
        }
        used += got;
    }
    data[used] = '\0';
    CloseHandle(hRead);

    /* Программа не может быть завершена, пока не завершены все
```

```
дочерние. */
    if (created > 0) {
        WaitForMultipleObjects((DWORD)created, procs, TRUE,
INFINITE);
    }
    for (int i = 0; i < created; i++) CloseHandle(procs[i]);
    free(procs);

    int *all = malloc(MAX_NUMBERS * sizeof(int));
    if (!all) {
        fprintf(stderr, "Ошибка: не удалось выделить память\n");
        return 1;
    }
    int total_found = 0;

    /* Разбор строк CHUNK <PID> <количество> 0 <числа...> */
    char *line = data;
    while (line && *line) {
        char *nl = strchr(line, '\n');
        if (nl) *nl = '\0';
        if (strncmp(line, "CHUNK ", 6) == 0) {
            char *p = line + 6;
            (void)strtol(p, &p, 10);
            int count = (int)strtol(p, &p, 10);
            (void)strtol(p, &p, 10);
            for (int i = 0; i < count && total_found < MAX_NUMBERS;
i++) {
                all[total_found++] = (int)strtol(p, &p, 10);
            }
        }
        line = nl ? nl + 1 : NULL;
    }
    free(data);

    double t1 = now_seconds();

    printf("Число M: %d\n", m);
    printf("Диапазон [L, R]: [%d, %d]\n", l, r);
    printf("Количество процессов K: %d\n", k);
    printf("Общее количество найденных чисел: %d\n", total_found);
    printf("Найденные числа:");
    for (int i = 0; i < total_found; i++) printf(" %d", all[i]);
    printf("\n");
    printf("Время работы: %.6f c\n", t1 - t0);

    free(all);

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.11 – *Lab*-03*d-server.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>
#include <string.h>

#include "console_ru.h"

#define PHRASE "Hello, my dear child!"

static void child_path(char *out, size_t cap)
{
    char self[MAX_PATH];
    GetModuleFileNameA(NULL, self, MAX_PATH);
    char *slash = strrchr(self, '\\');
    if (slash != NULL) *(slash + 1) = '\0';
    else self[0] = '\0';
    snprintf(out, cap, "%sLab-03hello-child.exe", self);
}

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(void)
{
    int pause = should_pause();

    char child[MAX_PATH];
    child_path(child, sizeof(child));

    printf("Родитель: передаю дочернему процессу через STARTUPINFO
строку:\n");
    printf("          \"%s\"\n", PHRASE);

    /* Фраза уходит в поле lpTitle; Windows применяет его как
заголовок окна
       только при создании новой консоли (CREATE_NEW_CONSOLE). */
    STARTUPINFOA si = {0};
    si.cb = sizeof(si);
    si.lpTitle = (LPSTR)PHRASE;

    PROCESS_INFORMATION pi = {0};

    if (!CreateProcessA(child, NULL, NULL, NULL, FALSE,
CREATE_NEW_CONSOLE, NULL, NULL, &si, &pi)) {
```

```
        fprintf(stderr, "Ошибка: CreateProcess (%lu)\n",
GetLastError());
        return 1;
    }

    printf("Родитель: дочерний процесс создан, PID=%lu\n", (unsigned
long)pi.dwProcessId);
    printf("Родитель: фраза передана через STARTUPINFO (lpTitle).
\n");

    CloseHandle(pi.hProcess);
    CloseHandle(pi.hThread);

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.12 – *Lab*-03*hello-parent.c* (*Windows*)

```
#include <windows.h>
#include <stdio.h>

#include "console_ru.h"

static int should_pause(void)
{
    return GetEnvironmentVariableA("PAUSE_ON_EXIT", NULL, 0) != 0;
}

static void pause_before_exit(void)
{
    printf("PID=%lu: нажмите Enter для выхода...\n", (unsigned
long)GetCurrentProcessId());
    fflush(stdout);
    getchar();
}

int main(void)
{
    int pause = should_pause();

    /* Единственный источник данных – заголовок консоли, который
родитель
       задал через STARTUPINFO.lpTitle. */
    char title[256];
    DWORD n = GetConsoleTitleA(title, sizeof(title));

    if (n == 0) {
        fprintf(stderr, "Дочерний процесс: не удалось прочитать
заголовок консоли (%lu)\n", GetLastError());
        return 1;
    }

    printf("Дочерний процесс (PID=%lu): получено через STARTUPINFO:
%s\n",
           (unsigned long)GetCurrentProcessId(), title);
    fflush(stdout);

    Sleep(4000);

    if (pause) pause_before_exit();
    return 0;
}
```

Листинг А.13 – *Lab*-03*hello-child.c* (*Windows*)

```
/*
 * console_ru.h – корректный вывод кириллицы в консоль cmd (Windows).
 *
 * Исходные файлы хранятся в UTF-8 и собираются с ключом /utf-8,
поэтому
 * строковые литералы в программе – UTF-8. Консоль cmd работает в
OEM-кодировке
 * (866), поэтому весь вывод перекодируется из UTF-8 в 866 через
 * MultiByteToWideChar + WideCharToMultiByte.
 *
 * Подключается после <stdio.h>. Заменяет printf/fprintf на варианты
 * rprintf/rfprintf, которые перекодируют уже сформированную строку
вывода.
 * ASCII-данные (пути, идентификаторы, числа) при перекодировке не
меняются.
 */
#ifndef CONSOLE_RU_H
#define CONSOLE_RU_H

#include <windows.h>
#include <stdio.h>
#include <stdarg.h>

/* UTF-8 -> OEM-кодировка консоли (866); буферы чередуются между
вызовами */
static const char *utf8_to_oem(const char *src)
{
    static char buf[4][4096];
    static int idx = 0;
    char *out = buf[idx];
    idx = (idx + 1) & 3;

    int wlen = MultiByteToWideChar(CP_UTF8, 0, src, -1, NULL, 0);
    if (wlen <= 0 || wlen > 4096) {
        return src; /* на случай неожиданных данных */
    }
    wchar_t wbuf[4096];
    MultiByteToWideChar(CP_UTF8, 0, src, -1, wbuf, wlen);
    int olen = WideCharToMultiByte(866, 0, wbuf, -1, out, 4096, NULL,
NULL);
    if (olen <= 0) {
        return src;
    }
    return out;
}

static int rprintf(const char *fmt, ...)
{
    char tmp[4096];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(tmp, sizeof(tmp), fmt, ap);
    va_end(ap);
```

```
    fputs(utf8_to_oem(tmp), stdout);
    fflush(stdout);
    return 0;
}

static int rfprintf(FILE *stream, const char *fmt, ...)
{
    char tmp[4096];
    va_list ap;
    va_start(ap, fmt);
    vsnprintf(tmp, sizeof(tmp), fmt, ap);
    va_end(ap);
    fputs(utf8_to_oem(tmp), stream);
    fflush(stream);
    return 0;
}

#define printf  rprintf
#define fprintf rfprintf

#endif /* CONSOLE_RU_H */
```

Листинг А.14 – *console_ru.h* (*Windows*)